

RENANCE

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

CSC 315 — Past Questions & Materials
by Renance

Course: CSC 315
Institution: Federal University of Petroleum Resources, Effurun
Papers in this pack: 5
Levels: 300

This study pack was compiled and restructured by Renance for CSC 315 students at Federal University of Petroleum Resources, Effurun. It brings together 5 past paper(s)/material(s) into one clean, readable document so you can revise without chasing scattered PDFs. Practise these same questions interactively on Renance with instant answers, AI explanations, and full CBT mock exams — over 235,000+ questions across JAMB, Post-UTME, WAEC, NECO and university courses.

Study online: renance.app
Get the app: Google Play — [dev.renance.renance](https://play.google.com/store/apps/details?id=dev.renance.renance)

Generated by Renance AI

Paper 1 of 5 — CSC 315 Algorithm and Complexity Analysis 2024/2025.

Level 300 • First Semester • PastQuestion

DEPARTMENT OF COMPUTER SCIENCE

: COLLEGE OF SCIENCE

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

2024/2025 FIRST SEMESTER EXAMINATION

CSC315 [ALGORITHMS & COMPLEXITY ANALYSIS]

INSTRUCTIONS:

(i) Answer question number one and any other three (iv) All extra sheets used should be signed by the Invigilator

questions (v) Submit your question paper along with your answer

Gi) Time Allowed: 2Hrs: 10mins booklet

(iii) Do not mix up questions. Start a new question on a fresh page.

Question One (Compulsory)

a) A supermarket gives a discount of 15% for purchases greater than or equal to N5,000 (five thousand naira). For any other

purchase amount, the discount is 5%. A customer gets discount only if payment was by cash. An algorithm is to be designed to

compute a customer's bill given the amount purchased as input. The algorithm is to calculate and output for at least 2000

customers. You are required to design the algorithm using pseudo code for the programming problem and state the complexity

class the algorithm belongs. (12mks)

b) Give three set back of flowchart as algorithmic tool. (6 mks)

Question Two

a) Using pseudo code only, illustrate how two numbers that are out of order could be swapped in a sorting problem (6mks)

b) Given $T(n) = 2n^2 + 5n + 2n + 5$. Give the run time of the algorithm and determine the percentage contribution of each term in

the expression. (6mks) Take $n = 15$

c) Explain succinctly why the first algorithm in the following pairs outperforms the second algorithm (6mks)

(i) Selection sort and bubble sort

qi) Insertion sort and Selection sort

Question Three

a) Contrast the basic idea of Prim and Kruskal's algorithms and explain how the two algorithms exhibit the optimal substructure

property (7mks)

b) in a tabular form only, use the Prim's algorithm to compute the weight/cost of the following minimum spanning tree (7mks)

28

22

ce) Using your table derived in (b) above, construct the final output of the given minimum spanning tree (4mks)

Question Four

a) Enumerate any four approaches for handling the activity selection problem (8mks)

b) The following list of courses, with their start and finish time enclosed in parenthesis, are to be scheduled for TLH1 lecture hall

on a single day, determine the possible sets of mutually compatible courses that should be scheduled using earliest finish time

algorithm (10mks) — CSC101 (0,6), CSC102(8,11), CSC103(12,14), CSC211(3,5), CSC212(6,10), CSC311(5,9), CSC312(1,4),

CSC313(8,12), CSC315(5,7), CSC411(3,8), CSC414(2,13).

Question Five

a) Enumerate three parameters that could be considered in computing the time complexity of algorithms (6mks)

b)

i). Explain succinctly why the worst-case efficiency is often used in algorithm analysis rather than best case efficiency (3mks)

ii). Give two possible worst-case scenarios in a linear search algorithm: (4mks)

c) The choice of input size greatly depends on primitive operations. In not more than

one sentence, explain the term "primitive

operations' and give three examples of such operations (3mks)

Question Six

a) i). Explain three possible reasons why software developers and designers would want to know the time efficiency of their

algorithms (6mks)

ii). Give two components of space complexity analysis of algorithms (4mks)

~ b) Profiling is one of the methods for estimating the run time of algorithms. Briefly describe the method and state three limitations.

of the method (8mks)

Paper 2 of 5 — CSC 315 PAST QUESTION

Level 300 • First Semester • PastQuestion

(€) Using your table derived in (b) above, construct the final output

, oft

\eondiaysraon utp he given minimum

if

4) Explain the term greedy algorithm and give two properties of a problem solved

algorithm (6mks) =

(i) Give three applications of greedy algorithms (3mks)

a Enumerate any four approaches for handling the activity selection problem (4mks)

Given the table below, determine the possible sets of mutually compatible activities using any of the approaches you enumerated in (bi) above (5mks)

rR ie 2 3 aS le [se [8 ie aan

Si 3 3 0 5 3 5 6 8 Si Lolli

[fi] 4 5 6 8 8 ECR RES ERE

fa) Enumerate three parameters that could be considered in computing the time complexity of algorithms (6mks)

(b) Explain succinctly why the worst case efficiency is often used in algorithm analysis rather Give two possible worst case scenarios in a linear search algorithm (4mks)

input size greatly depends on primitive operations. In not more than one

the term 'primitive operations' and give three examples of such

Paper 3 of 5 — CSC315 ASSIGNMENT

Level 300 • First Semester • Handout

CSC315 ASSIGNMENT

A supermarket gives a discount of 25% for purchases greater than or equal to N15,000 (fifteen thousand naira). For any other purchase amount, the discount is 15%. A customer gets discount only if payment was by cash. An algorithm is to be designed to compute a customer's bill given the amount purchased as input. The algorithm is to calculate and output for at least 5000 customers. You are required to:

- i. Design the algorithm using pseudo code for the programming problem
- ii. Analyze and determine the class of the algorithm you have designed

To be submitted individually on Tuesday March 25th 2025

Paper 4 of 5 — CSC315 ALGORITHMS & COMPLEXITY ANALYSIS

[Orhionkpaiyo, B. C]

Level 300 • First Semester • Handout

Department of Computer Science

Federal University of Petroleum Resources, Effurun,

Delta State

CSC315

ALGORITHMS & COMPLEXITY ANALYSIS

[Orhionkpaiyo, B. C]

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Course Aim:

Algorithms (along with data structures) are the fundamental “building blocks” from which programs are constructed. Only by fully understanding them is it possible to write very effective programs. An algorithmic solution to a computational problem will usually involve designing an algorithm, and then analysing its performance. Algorithmic is a branch of computer science that consists of designing and analyzing computer algorithms. Therefore, this course introduces the basics of the analyses of time and space complexity of some basic algorithms and the notations for describing them. The goal is to provide students with the fundamental knowledge of common algorithms and to deal with a wide variety of computational problems.

Objectives

- i. Describe concepts of algorithm, algorithm design and analysis
- ii. Demonstrate knowledge of algorithm complexity analysis
- iii. Categorize and describe some specific algorithms
- iv. Determine the running time of the basic algorithms in different domains
- v. Analyze the asymptotic performance of algorithms
- vi. Apply important algorithmic design paradigms and methods of analysis.
- vii. Determine worst-case running times of algorithms using asymptotic analysis.
- viii. Determine asymptotic growth rates of algorithms.
- ix. Design and implement efficient divide and conquer algorithms suitable to solve a particular computational problem
- x. Compare algorithms with respect to time and space complexity
- xi. Describe the relative merits of worst, average, and best case analysis of algorithms.
- xii. Analyze recursive and non-recursive algorithms
- xiii. Solve recurrence problems using the master theorem

Compiled: November 2018 Last update: January 2023 Page 2

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Course Outline

1. Fundamentals of Algorithms and Complexity Analysis

- i. Meaning and characteristics of algorithm
- ii. Methods of expressing algorithms
- iii. Basis for studying algorithm

2. Standard Complexity Classes

- o Algorithm Complexity
- o Methods of estimating the runtime of algorithms
- o Asymptotic notations
- o Runtime classification of algorithms
- o Time & Space trade-offs in Algorithm Analysis

3. Non-recursive Algorithms

i. Fundamental Computing Algorithms

- o Linear search
- o Binary Search
- o Bubble sort
- o Selection sort
- o Insertion sort

4. Greedy Algorithms

i. General methods and characteristics of Greedy algorithms

ii. Application of Greedy algorithms

- o Activity selection problem
- o Minimum spanning tree
- o Knapsack problem
- o Unit task scheduling with deadline

5. Divide-and-Conquer Algorithm Design Strategies

i. Quicksort

ii. Binary searchtree

Compiled: November 2018 Last update: January 2023 Page 3

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Textbooks:

1. Introduction to Algorithms – Thomas H. Cormen, Charles E. Leiserson and Ronald L. Rivest
2. Introduction to the Design and Analysis of Algorithms – Anany Levitin

Contacts:

Grading:

- (i) Assignments-----12mks (or 0 for copying)
- (ii) Attendance.....3mks (70% total attendance)
- (iii) Test-----15mks (or 0 for cheating)
- (iv) Exam-----70mks
- (v) Total-----100%

Note: Late assignment submission will be graded as mark - x, where x is the number of days of late submission.

Compiled: November 2018 Last update: January 2023 Page 4

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Lecture Schedule

Week Topic

1 Students' registration

2 Fundamentals of Algorithms and Complexity Analysis

- i. Meaning and characteristics of algorithm
- ii. Methods of expressing algorithms
- iii. Basis for studying algorithm

3-5 Standard Complexity Classes

- o Algorithm Complexity
- o Methods of estimating the runtime of algorithms
- o Asymptotic notations
- o Runtime classification of algorithms
- o Time & Space trade-offs in Algorithm Analysis

6-8 Non-recursive Algorithms

- i. Fundamental Computing Algorithms
 - o Linear search
 - o Binary Search
 - o Bubble sort
 - o Selection sort
 - o Insertion sort

9 - 11 Greedy Algorithms

- i. General methods and characteristics of Greedy algorithms
- ii. Application of Greedy algorithms
 - o Activity selection problem
 - o Minimum spanning tree
 - o Knapsack problem
 - o Unit task scheduling with deadline

12 Divide-and-Conquer algorithm Design Strategies

- i. Quicksort
- ii. Binary search tree

13 Test and revision

14 – 15 Examination

Compiled: November 2018 Last update: January 2023 Page 5

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

1. Basic Concepts of Algorithms

1.1 What is Algorithm?

□ An algorithm is a finite sequence of steps expressed for solving a problem.

□ An algorithm can be defined as “a process that performs some sequence of operations in order to solve a given problem”.

□ Algorithm is a step-by-step procedure, which defines a set of instructions to be executed in a certain order to get the desired output.

A procedure can only be called an algorithm to solve a particular problem if it has the following characteristics.

- i. Independence – An algorithm should have step-by-step directions, which should be independent of any programming code.
- ii. Feasibility – It should be feasible within the available resources.
- iii. Input/Output – An algorithm should have 0 or more well-defined inputs and one or more well-defined outputs.
- iv. Finiteness – Algorithm must terminate after a finite number of steps
- v. Unambiguous – Algorithm should be clear and unambiguous. Each step (or phase) and its input/output should be clear and must lead to only one meaning.

Algorithms are used for calculation, data processing, and many other fields.

Algorithms are generally created independent of underlying languages, i.e. an algorithm can be implemented in more than one programming language.

□ Benefits of using Algorithms

The use of algorithms provides a number of benefits.

□ One of these benefits is in the development of the procedure itself, which involves identification of the processes, major decision points, and variables necessary to solve the problem. Identification of the processes

Compiled: November 2018 Last update: January 2023 Page 6

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

and decision points reduces the task into a series of smaller steps of more manageable size.

□ Problems that would be difficult or impossible to solve in entirety can be approached as a series of small, solvable sub-problems.

□ An algorithm serves as a mnemonic device and helps ensure that variables or parts of the problem are not ignored. Presenting the solution process as an algorithm allows more precise communication. Finally,

separation of the procedure steps facilitates division of labour and development of expertise.

□ A final benefit of the use of an algorithm comes from the improvement it makes possible. As results are compared with goals overtime, the existence of a specified solution process allows identification of weaknesses and errors in the process.

o Reduction of a task to a specified set of steps or algorithm is an important part of analysis, control and evaluation.

1.2 Reasons for Using Algorithm

Three reasons for using algorithms are efficiency, abstraction and reusability.

□ Efficiency: Certain types of problems, like sorting, occur often in computing. Efficient algorithms must be used to solve such problems considering the time and cost factor involved in each algorithm.

□ Abstraction: Algorithms provide a level of abstraction in solving problems because many seemingly complicated problems can be distilled into simpler ones for which well known algorithms exist. Once we see a more complicated problem in a simpler light, we can think of the simpler problem as just an abstraction of the more complicated one.

o For example, imagine trying to find the shortest way to route a packet between two gateways in an internet. Once we realize that

Compiled: November 2018 Last update: January 2023 Page 7

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

this problem is just a variation of the more general shortest path problem, we can solve it using the generalised approach.

□ Reusability: Algorithms are often reusable in many different situations. Since many well-known algorithms are the generalizations of more complicated ones, and since many complicated problems can be distilled into simpler ones, an efficient means of solving certain simpler problems potentially lets us solve many complicated problems.

1.3 Methods of Expressing Algorithms

Algorithms can be expressed in many different notations, including

- o natural languages,
- o pseudocode,
- o flowcharts and
- o Programming languages

Natural language expressions of algorithms tend to be verbose and ambiguous, and are rarely used for complex or technical algorithms.

Pseudocode and flowcharts are structured ways to express algorithms that avoid many ambiguities common in natural language statements, while remaining

independent of a particular implementation language.

Pseudocode is a mixture of a natural language and programming language-like constructs. Pseudocode is usually more precise than natural language, and its usage often yields more succinct algorithm descriptions. The running time of an algorithm can be estimated in a more general manner by using Pseudocode to represent the algorithm as a set of fundamental operations which can then be counted.

Flowcharts are used in analyzing, designing, documenting or managing a process or program in various fields. The purpose of all flow charts is to

Compiled: November 2018 Last update: January 2023 Page 8

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

communicate how a process works or should work without any technical or group specific jargon.

Programming languages are primarily intended for expressing algorithms in a form that can be executed by a computer, but are often used to define or document algorithms.

Sometimes it is helpful in the description of an algorithm to supplement small flowcharts with natural language and/or arithmetic expressions written inside block diagrams to summarize what the flowcharts are accomplishing.

□ Some examples of how to express Algorithms

1. Algorithm to sum the numbers between 1 and 10 inclusive. This is represented in three forms – natural language, pseudo code and flowchart

A: Natural Language

1. Initialize sum
2. Repeat steps 3 and 4 ten times
3. Read number
4. Add number to sum
5. Print sum

Compiled: November 2018 Last update: January 2023 Page 9

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

B. Pseudo code

Input: Integer numbers between 1 and 10 inclusive

Output: Sum of ten integer numbers

1. $Sum \leftarrow 0$
2. for $i \leftarrow 1$ to 10
3. Read number
4. $Sum \leftarrow sum + i$
5. end for
6. Print Sum

C. Flowchart Representation

Start

Read Num

Sum = sum + Num

Yes

More?

No

Print Sum

Stop

Compiled: November 2018 Last update: January 2023 Page 10

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

2. Finding the largest element in array

A. Using Natural language

1. Assign the first element of the array as the largest
2. Repeat step 3 starting from the second element to the last element in the array
3. Swap, if the next element is greater than the element in the previous position

B. Using Pseudocode

Algorithm Max (A, n)

Input: An array A of n integer

Output: Largest element in A

1. Largest $\leftarrow A[1]$
2. For $i \leftarrow 2$ to n do
3. if largest $< A[i]$ then largest $\leftarrow A[i]$
- 4 End for

Pseudo-code provides a way of expressing algorithms in a way that is independent of any programming language. It abstracts away other program details such as the type system and declaring variables and arrays. Some points to note are:

- The statement blocks are determined by indentation, rather than {and} delimiters as in Java.
- Control statements, such as if...then...else and while have similar interpretations to Java.
- A statement $v \leftarrow e$ implies that expression e should be evaluated and the resulting value assigned to variable v . Or, in the case of $v1 \leftarrow v2 \leftarrow e$, to

variables v1 and v2

Compiled: November 2018 Last update: January 2023 Page 11

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

- All variables should be treated as local to their procedures.
- Arrays indexation is denoted by $A[i]$ and arrays are assumed to be indexed from 1 to N (rather than 0 to $N - 1$, the approach followed by Java).

1.4 Why Study Algorithm?

i. Computers may be fast, but they are not infinitely fast. And memory may be inexpensive, but it is not free. Computing time is therefore a bounded resource, and so is space in memory. You should use these resources wisely, and algorithms that are efficient in terms of time or space will help do so.

ii. Algorithms are at the core of most technologies used in contemporary computers. Even an application that does not require algorithmic content at the application level relies heavily upon algorithms.

a. Does the application rely on fast hardware? The hardware design used algorithms.

b. Does the application rely on graphical user interfaces? The design of any GUI relies on algorithms.

c. Does the application rely on networking? Routing in networks relies heavily on algorithms.

d. Was the application written in a language other than machine code? Then it was processed by a compiler, interpreter, or assembler, all of which make extensive use of algorithms.

iii. Furthermore, with the ever-increasing capacities of computers, we use them to solve larger problems than ever before. It is at larger problem sizes that the differences in efficiency between algorithms become particularly prominent.

iv. Having a solid base of algorithmic knowledge and technique is one characteristic that separates the truly skilled programmers from the

Compiled: November 2018 Last update: January 2023 Page 12

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

novices. With modern computing technology, you can accomplish some tasks without knowing much about algorithms, but with a good background in algorithms, you can do much, much more.

Exercises

1. A program to find the n th term of an arithmetic progression is to be written, given the first term (a) and the common difference (d). Write the pseudo code for the program.

2. A supermarket gives a discount of 15% for purchases greater than or equal to five thousand naira. For any other purchase amount, the discount is 5%.

A program is to be written to compute a customer's bill given the amount purchased as input. The program is to calculate and output for at least 2000 customers and a customer gets discount only if payment was by cash. Write a pseudo code for the programming problem.

3. Write an algorithm to compute the sum of the squares of integers from 1 to 50 and also draw the corresponding flowchart. Assuming that you have to code the above given problem, show the steps involved in program design.

Compiled: November 2018 Last update: January 2023 Page 13

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

2. Standard Complexity Classes

Analysis of algorithms is the theoretical study of computer program performance and resource usage, and is often practised abstractly without the use of specific programming language or implementation.

□ The practical goal of algorithm analysis is to predict the performance of different algorithms in order to guide program design decisions.

Inefficient Algorithms can significantly impact system performance.

o In time-sensitive applications, an algorithm taking too long to run can render its results outdated or useless.

o An inefficient algorithm can also end up requiring an uneconomical amount of computing power or storage in order to run, again rendering it practically useless.

2.1 Algorithm Complexity

The complexity (Computational complexity) of an algorithm describes the efficiency of the algorithm in terms of the amount of the memory required to process the data and the processing time. Algorithmic complexity is concerned about its performance, how fast or slow it works.

□ Most algorithms do not perform the same in all cases; normally an algorithm's performance varies with the data passed to it.

Complexity of an algorithm is analyzed in two perspectives: time and space.

(i) Time Complexity

It's a function describing the amount of time required to run an algorithm in terms of the size of the input.

Compiled: November 2018 Last update: January 2023 Page 14

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

"Time" can mean the number of memory accesses performed, the number of comparisons between integers, the number of times some inner loop is

executed, or some other natural unit related to the amount of real time the algorithm will take.

For a given problem, we characterize the input size, n , appropriately:

- o Sorting – The number of items to be sorted
- o Graphs – The number of vertices and/or edges
- o Numerical – The number of bits needed to represent a number

The choice of an input size greatly depends on the elementary operation; the most relevant or important operation of an algorithm.

- o Comparisons
- o Additions
- o Multiplications

Indeed, for small values of n , most functions will be very similar in running time. Only for sufficiently large n do differences in running time become apparent.

Types of Complexity

□ Worst-case complexity: The worst-case efficiency of an algorithm is its efficiency for the worst-case input of size n , which is an input (or inputs) of size n for which the algorithm runs the longest among all possible inputs of that size.

Worst-case analysis answers questions such as - what is the longest an algorithm can run for a problem of size n ?

Or, among all inputs of the same size, what is the maximum running time? Computer scientists mostly use worst-case analysis

Compiled: November 2018 Last update: January 2023 Page 15

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The worst-case analysis provides very important information about an algorithm's efficiency by bounding its running time from above. In other words, it guarantees that for any instance of size n , the running time will not exceed $C \text{ worst}(n)$ it's running time on the worst-case inputs.

For example, in a linear search, the worst case is when the target item is at the end of the list or not in the list at all. Then the algorithm must visit every item and perform n iterations for a list of size n . Thus, the worst-case complexity of a linear search is $O(n)$.

There are four reasons why algorithms are generally analyzed by their worst case:

□ Many algorithms perform to their worst case a large part of the time. For example, the worst case in searching occurs when we do not find what we are looking for at all. This frequently happens in database

applications.

□ The best case is not very informative because many algorithms perform exactly the same in the best case. For example, nearly all searching algorithms can locate an element in one inspection at best, so analyzing this case does not tell us much.

□ Determining average-case performance is not always easy. Often it is difficult to determine exactly what the “average case” even is; since we can seldom guarantee precisely how an algorithm will be exercised.

□ The worst case gives us an upper bound on performance. Analyzing an algorithm’s worst case guarantees that it will never perform worse than what we determine. Therefore, we know that the other cases must perform at least better than the worst case.

Compiled: November 2018 Last update: January 2023 Page 16

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

□ Average-case complexity: Among all inputs of the same size, what is the average running time? The average is computed assuming some probability distribution that describes the likelihood that a particular input will arise.

Average-case answers questions such as - on average how fast does an algorithm run for a problem of size N ?

To determine the average case, for example in a linear search, add the number of iterations required to find the target at each possible position and divide the sum by n . Thus, the algorithm performs $(n + n - 1 + n - 2 + \dots + 1) / n$, or

$(n+1)/2$ iterations.

For very large n , the constant factor of $/2$ is insignificant, so the average complexity is still $O(n)$ as in worst case.

The best-case efficiency of an algorithm is its efficiency for the best-case input of size n , which is an input (or inputs) of size n for which the algorithm runs the fastest among all possible inputs of that size.

Best-case analysis answers the question such as - what is the fastest an algorithm can run for a problem of size N ?

For example, in a linear search, the best case is when the algorithm finds the target at the first position, after making one iteration, for an $O(1)$ complexity.

□ Why know Time efficiency of algorithms?

We like to know the time efficiency of a program for several reasons:

- o To get an estimate of how long a program will run. (Waiting for it

to finish may not be feasible!)

Compiled: November 2018 Last update: January 2023 Page 17

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

- o To get an estimate of how large an input the program can handle without taking too long.

- o To compare the efficiency of different programs for solving the same problem.

(ii) Space Complexity

It's a function describing the amount of memory an algorithm takes in terms of the size of input to the algorithm. We often speak of "extra" memory needed, not counting the memory needed to store the input itself. Again, we use natural (but fixed-length) units to measure this.

Space complexity is sometimes ignored because the space used is minimal and/or obvious, however sometimes it becomes as important an issue as time.

Space is measured by counting the maximum memory space required by the algorithm. The space required by an algorithm is equal to the sum of the following two components:

- A fixed part that is a space required to store certain data and variables that are independent of the size of the problem. For example, simple variables and constants used, program size, etc.
- A variable part is a space required by variables, whose size depends on the size of the problem. For example, dynamic memory allocation, recursion stacks space, etc.

Compiled: November 2018 Last update: January 2023 Page 18

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

2.2 Methods of estimating the Runtime of algorithm

a. Use of computer Clock

One way to measure the time cost of an algorithm is to use the computer's clock to obtain an actual run time. This process is called benchmarking or profiling

- o It starts by determining the time for several different data sets of the same size and then calculates the average time.

- o Next, similar data are gathered for larger and larger data sets. After several such tests, enough data are available to predict how the algorithm will behave for a data set of any size.

The program uses the time() function to track the running time.

This function returns the number of seconds that have elapsed between the current time on the computer's clock and start time.

Thus, the difference between the results of two calls of `time.time()` represents the elapsed time in seconds.

e.g.

```
Start = time.timer()
```

```
//start of the algorithm
```

```
The algorithm
```

```
//End of the algorithm
```

```
Elapsed = time.timer - start
```

This method permits accurate predictions of the running times of many algorithms.

However, there are some problems with this technique:

- Different hardware platforms have different processing speeds, so the running times of an algorithm differ from machine to machine.

Compiled: November 2018 Last update: January 2023 Page 19

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

- Also, the running time of a program varies with the type of operating system that lies between it and the hardware.

- Finally, different programming languages and compilers produce code whose performance varies. For example, an algorithm coded in C usually runs slightly faster than the same algorithm in Python byte code.

- The “coding efficiency” of the programmer who converts the algorithm to a program can have a tremendous impact as well.

- It is impractical to determine the running time for some algorithms with very large data sets. For some algorithms, it doesn’t matter how fast the compiled code or the hardware processor is. They are impractical to run with very large data sets on any computer.

Thus, predictions of performance generated from the results of timing on one hardware or software platform generally cannot be used to predict potential performance on other platforms.

So it is not an independent measure of the algorithm, but rather a measure of the implementation, the machine and the instance.

b. Counting Instructions

Another technique used to estimate the efficiency of an algorithm is to count the instructions executed with different problem sizes. These counts provide a good predictor of the amount of abstract work performed by an algorithm, no matter what platform the algorithm runs on.

Keep in mind, however, that when you count instructions, you are counting the instructions in the high-level code in which the algorithm is

written, not instructions in the executable machine language program.

Compiled: November 2018 Last update: January 2023 Page 20

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

When analyzing an algorithm in this way, you distinguish between two classes of instructions:

- i. Instructions that execute the same number of times regardless of the problem size
- ii. Instructions whose execution count varies with the problem size
e.g. instructions within loops

Note: The first class of instructions does not affect significantly in algorithm analysis. The instructions in the second class normally are found in loops or recursive functions. In the case of loops, focus is also on instructions performed in any nested loops or, more simply, just the number of iterations that a nested loop performs.

Of primary consideration when estimating an algorithm's performance is the number of basic operations required by the algorithm to process an input of a certain size.

□ Size is often the number of inputs processed. For example, when comparing sorting algorithms, the size of the problem is typically measured by the number of records to be sorted.

□ A basic operation must have the property that its time to complete does not depend on the particular values of its operands.

o Adding or comparing two integer variables are examples of basic operations in most programming languages.

o Summing the contents of an array containing n integers is not, because the cost depends on the value of n (i.e., the size of the input).

Primitive (basic) operations include:

- Calling a method and returning from a method
- Performing an arithmetic operation (e.g. addition)

Compiled: November 2018 Last update: January 2023 Page 21

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

- Comparing two numbers
- Assigning a value to a variable
- Indexing an array
- Evaluating an expression, etc

Because the most important factor affecting running time is normally size of the input, for a given input size n we often express the time T to run the algorithm as a function of n , written as $T(n)$.

Time is measured by counting the number of key operations such as comparisons in a sorting algorithm.

c. Asymptotic complexity measure

Time of algorithm can also be measured without necessarily counting the exact number of steps of a program, but how that number grows with the size of the input to the program. This approach is based on the asymptotic complexity measure.

The limiting behaviour of the complexity as size increases is called the asymptotic time complexity.

It is the asymptotic complexity of an algorithm, which ultimately determines the size of problems that can be solved by the algorithm. The asymptotic complexity is written using the appropriate asymptotic notations discussed in the next section.

Compiled: November 2018 Last update: January 2023 Page 22

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

2.3 Asymptotic Notations

Asymptotic analysis refers to the study of an algorithm as the input size “gets big” or reaches a limit. Asymptotic notation gives us a method for classifying functions according to their rate of growth.

It allows the analysis of an algorithm’s running time by identifying its behaviour as the input size for the algorithm increases. This is also known as an algorithm’s growth rate.

Asymptotic analysis attempts to:

- o estimate the resource consumption of an algorithm
- o compare the relative costs of two or more algorithms for solving the same problem.

Asymptotic analysis also gives algorithm designers a tool for estimating whether a proposed solution is likely to meet the resource constraints for a problem before they implement an actual program.

Let us imagine an algorithm as a function f , n as the input size, and $f(n)$ being the running time. So for a given algorithm f , with input size n you get some resultant run time $f(n)$. This can be represented in a graph where the y axis is the runtime, x axis is the input size, and plot points are the resultants of the amount of time for a given input size.

Compiled: November 2018 Last update: January 2023 Page 23

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Primarily we are interested only in the growth rate of f , which describes how quickly the algorithm’s performance will degrade as the size of data it processes

becomes arbitrarily large.

(i) Upper bound: It indicates the upper or highest growth rate that the algorithm can have. A special notation, called big-Oh notation is adopted. It is usually written in relation to the input size such as $O(n)$. The $O(n)$ is read as "Oh of n " or "big Oh of n ". The $O(n)$ measures the worst case time complexity or the longest amount of time an algorithm can possibly take to complete.

The following is a precise definition for an upper bound:

For $f(n)$ a non-negatively valued function, $f(n)$ is in set $O(g(n))$ if there exist two positive constants c and n_0 such that $f(n) \leq cg(n)$ for all $n > n_0$.

In other words,

$f(n)$ is said to be $O(g(n))$

if there exists a natural number n_0 and a constant $c > 0$ such that

$\forall n \geq n_0; f(n) \leq cg(n)$:

The definition says that for all inputs of the type in question (such as the worst case for all inputs of size n) that are large enough (i.e., $n > n_0$), the algorithm always executes in less than $cg(n)$ steps for some constant c .

NOTE: i. $f(n)$ represents the true running time of the algorithm.

ii. $g(n)$ is some expression for the upper bound (i.e. $g(n)$ is an arbitrary time complexity you are trying to relate to your algorithm).

iii. Constant n_0 is the smallest value of n for which the claim of an upper bound holds true

Compiled: November 2018 Last update: January 2023 Page 24

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

iv. c encapsulates many factors eg. Implementation details of algorithm, machine, technology used etc

This can be represented graphically as shown below

It should be emphasized, however, that this does not mean that the running time is always as large as $cg(n)$, even for large input sizes. Thus, the O -notation provides an upper bound on the running time

Rules for using big- O :

The most important property is that big- O gives an upper bound only. If an algorithm is $O(n^2)$, it doesn't have to take n^2 steps (or a constant multiple of n^2). But it can't take more than n^2 . So any algorithm that is $O(n)$, is also an $O(n^2)$ algorithm. Think of big- O as being like " $<$ ". Any number that is $< n$ is also $< n^2$.

When faced with a complicated function like $3n^2 + 4n + 5$, we just replace it with $O(f(n))$, where $f(n)$ is as simple as possible. In this particular example we'd use $O(n^2)$, because the quadratic portion of the sum dominates the rest of the function.

Here are some rules that help simplify functions by omitting dominated terms:

Compiled: November 2018 Last update: January 2023 Page 25

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

□ We can ignore constant terms because as the value of n becomes larger and larger, eventually constant terms will become insignificant. For example, if $T(n) = n + 50$ describes the running time of an algorithm, and n , the size of data it processes, is only 1024, the constant term in this expression constitutes less than 5% of the running time.

□ We can ignore multiplicative constants because they too will become insignificant as the value of n increases. For example, $14n^2$ becomes n^2 .

□ We need to consider the highest-order term because as n increases higher-order terms quickly outweigh the lower-order terms. That is, na dominates nb if $a > b$. For instance, n^2 dominates n .

□ We also must note that an exponential term dominates any polynomial term in the expression. For example, 3^n dominates n^5 (it even dominates 2^n).

□ Similarly, a polynomial term dominates any logarithmic term used in the expression. For example, n dominates $(\log n)^3$. This also means, for example, that n^2 dominates $n \log n$.

Suppose we have an algorithm whose running time is described by the function $T(n) = 3n^2 + 10n + 10$.

Using the rules of O-notation, this function can be simplified to:

$$O(T(n)) = O(3n^2 + 10n + 10) = O(3n^2) = O(n^2)$$

We can verify this quantitatively by computing the percentage of the overall running time that each term accounts for as n increases. For example, when $n = 10$, we have the following:

$$\text{Running time for } 3n^2: 3(10)^2 / (3(10)^2 + 10(10) + 10) = 73.2\%$$

$$\text{Running time for } 10n: 10(10) / (3(10)^2 + 10(10) + 10) = 24.4\%$$

$$\text{Running time for } 10: 10 / (3(10)^2 + 10(10) + 10) = 2.4\%$$

Compiled: November 2018 Last update: January 2023 Page 26

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

This indicates that the term n^2 will be the one that accounts for the most of the running time as n grows arbitrarily large.

(ii) Lower Bounds: describe the least amount of a resource that an algorithm needs for some class of input (i.e. the worst-, average-, or best-case input of size n). The lower bound for an algorithm is denoted by the symbol Ω , pronounced “big-Omega” or just “Omega.” For example, $\Omega(n)$ is read as “Omega of n ” or “big-Omega of n ”. The notation $\Omega(n)$ measures the best case time complexity or the best amount of time an algorithm can possibly take to

complete.

For $f(n)$ a non-negatively valued function, $f(n)$ is in set $(g(n))$ if there exist two positive constants c and n_0 such that $f(n) \geq cg(n)$ for all $n > n_0$.

$\Omega(g(n)) = \{f(n): \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\}$

$\Omega(g(n))$ is the set of all functions with a larger or same order of growth as $g(n)$.

Compiled: November 2018 Last update: January 2023 Page 27

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

(iii) The big-Theta notation - Θ : When the upper and lower bounds are the same within a constant factor, we indicate this by using Θ (big-Theta) notation. $\Theta(g(n))$ is the set of all functions with the same order of growth as $g(n)$.

In general, we say that the running time of an algorithm is of order $\Theta(g(n))$ if whenever the input size is equal to or exceeds some threshold n_0 , its running time can be bounded below by $c_1g(n)$ and above by $c_2g(n)$, where $0 < c_1 \leq c_2$.

Thus, this notation is used to express the exact order of an algorithm, which implies an exact bound on its running time.

$\Theta(g(n)) = \{f(n): \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \leq c_1g(n) \leq f(n) \leq c_2g(n) \text{ for all } n \geq n_0\}$

These two constants - c_1 and c_2 - encapsulate many factors pertaining to the details of the implementation of the algorithm and the machine and technology used.

Represented graphically as:

It may be helpful to think of O as similar to $\leq$, Ω as similar to $\geq$ and Θ as similar to $=$.

2.4 Runtime Classification of Algorithms

The running time of an algorithm on a particular input is the number of primitive operations or “steps” executed.

Compiled: November 2018 Last update: January 2023 Page 28

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Constant time: An algorithm is said to be constant time (written as $O(1)$ time) if the value of $T(n)$ is bounded by a value that does not depend on the size of the input. This means that the algorithm requires the same fixed number of steps regardless of the size of the task.

For example, accessing any single element in an array takes constant time as only one operation has to be performed to locate it

Examples: (i):

(ii):

Note: Each of the assignment statements takes constant time. The for loop in the second example involves fixed number of operations and hence requires constant time.

More general examples include:

- array: accessing any element
- fixed-size stack: push and pop methods

Logarithmic ($\log n$): Implies that the running time of the algorithm is proportional to the logarithm of the input size. The algorithm of this type does not use the whole input.

Compiled: November 2018 Last update: January 2023 Page 29

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

o This occurs in programs that solve a big problem by transforming it into a smaller problem, cutting the size by some constant fraction.

o Whenever n doubles, $\log n$ increases by a constant, but $\log n$ does not double until n increases to n^2 . It always increases, but at a slower rate as n increases.

A logarithmic function is the inverse of an exponential function, i.e. $b^x = n$ is equivalent to

$$x = \log_b n$$

Logarithm denotes a quantity representing the power to which a fixed number (the base) must be raised to produce a given number. Thus

$$n = 2^k$$

is represented in logarithmic form as $\log_2 n = k$

What is important here is that the running time grows in proportion to the logarithm of the input (in this case, log to the base 2)

Consider the following example

```
Sum = 0;
for (k = 1; k <= n; k*=2) // do log n times.
sum ++;
```

If n is 8, the loop runs $\log(8)$ times i.e. 3 times.

More general examples of logarithm algorithm include

- o Binary search
- o Searching a tree data structure

Linear time (n): A linear-time algorithm is one that takes a number of steps directly proportional to the size of the input. In other words, if the size of the input doubles, the number of steps doubles. This means that the algorithm

requires a number of steps proportional to the size of the task. It is represented as $O(n)$ and referred to as Linear time.

Compiled: November 2018 Last update: January 2023 Page 30

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Basic structure is:

```
for (i = 0; i < N; i++) {
```

Sequence of statements of $O(1)$

The loop executes N times, so the total time is $N \cdot O(1)$ which is $O(N)$.

Examples:

(i)

```
for (int i = 0; i < data.Length; i++)
```

```
if (data[i] == find)
```

```
return i;
```

(ii):

(iii):

More general examples include

- o Traversing an array.

- o Sequential/Linear search in an array

Compiled: November 2018 Last update: January 2023 Page 31

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Quasi-linear or log-linear ($n \log n$): The running time grows in proportion to $n \log n$ of the input. It is a combination of $O(n)$ and $O(\log n)$ running time.

Example 1:

```
for (int i = 1; i <= n; i++){
```

```
for(int j = 1; j < 5; j = j * 2) {
```

```
System.out.println("i" + i + " and j" + j);
```

For example, if n is 8, then this algorithm will run $8 * \log(8) = 8 * 3 = 24$ times.

Example 2:

```
Sum = 0;
```

```
for (k = 1; k <= n; k*=2) // do log n times.
```

```
for (j = 1; j <= n; j++) //do n times
```

```
sum ++;
```

The inner loop executes n times while the outer loop executes $\log n + 1$ times because on each iteration k is multiplied by two until it reaches n . The total runtime is $n \log n$.

Quadratic time (n^2):. That is, the number of operations is proportional to the size of the task squared. It means the running time of the algorithm on an input

of size n is limited by the quadratic function of n . It is represented as $O(n^2)$ and referred to as quadratic time

- o Quadratic running times typically arise in algorithms that process all pairs of data items (perhaps in a double nested loop) whenever n doubles, the running time increases four fold.

- o When the running time of an algorithm is quadratic, it is practical for use only on relatively small problems

Compiled: November 2018 Last update: January 2023 Page 32

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Examples:

(i):

```
for (i = 0; i < N; i++) {
  for (j = 0; j < N; j++) {
    .....
  }
}
```

There are 2 nested for loops, each of which runs n times. The innermost loop therefore executes $n \times n = n^2$ times

(ii):

```
for (int i = 0; i < n; i++) // Loop1
  for (int j = i + 1; j > i; j--) // Loop2
    for (int k = n; k > j; k--) // Loop3
      system.out.println("*");
```

Loop2 runs a constant number of times (exactly once) every loop1 iteration and does not affect the time complexity (although there are three for loops, only two affect the running time, since they depend on the input size). We can carefully trace the values of j in each loop1/loop2 iteration to find the number of times the innermost loop iterates.

(iii):

Consider the following code:

What is the running time for this code fragment? Clearly it takes longer to run when n is larger.

Compiled: November 2018 Last update: January 2023 Page 33

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

- o The basic operation in this example is the increment operation for variable `sum`. The total number of increment operations is n^2 . Thus, we say that the running time is $T(n) = n^2$

Cubic (n^3): Similarly, an algorithm that process triples of data items (perhaps in a triple-nested loop) has a cubic running time. Whenever n doubles, the running time increases eight fold.

o It is practical for use only on small problems.

Example: The code to compute the matrix product $C = A * B$ is given below:

```
for (i = 1; i <= n; i++)
for (j = 1; j <= n; j++)
C[i, j] = 0;
for (k = 1; k <= n; k++)
C[i, j] = C[i, j] + A[i, k] * B[k, j];
```

There are 3 nested for loops, each of which runs n times. The innermost loop therefore executes $n * n * n = n^3$ times. The innermost statement, which contains a scalar sum and product takes constant $O(1)$ time. So the algorithm overall takes $O(n^3)$ time.

Exponential (2^n): Few algorithms with exponential running time are likely to be appropriate for practical use, such algorithms arise naturally as “brute-force” solutions to problems. Whenever n doubles, the running time squares.

Compiled: November 2018 Last update: January 2023 Page 34

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Polynomial (n^k): More generally, if an algorithm is $O(n^k)$ for constant k it is called a polynomial-time algorithm. Thus, quadratic and cubic running time algorithms are polynomial.

NOTE: There is difference between an exponential function and a polynomial. The function $p(x) = x^3$ is a polynomial. Here the “variable”, x , is raised to some constant power (i.e. 3 in the example). The function $f(x) = 3^x$ is an exponential function where the variable x is the exponent.

The graph below shows the growth of the algorithm running time with given input size. The growth rate for an algorithm is the rate at which the cost of the algorithm grows as the size of its input grows

Exercise

Using the given inputs in the table below, compute the growth rate of the three algorithms with $n \log n$, n^2 and $\log n$ runtime respectively and plot the graph to show the growth rate of the algorithm

Compiled: November 2018 Last update: January 2023 Page 35

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Input $\log n$

Size $n \log n$ n^2

256

1024

2048

4096

8192

Compiled: November 2018 Last update: January 2023 Page 36

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

3. Non-recursive Algorithms

A non-recursive algorithm or simply an iterative function is one that loops to repeat some part of the code; contrast to a recursive function that calls itself again to repeat the code.

o Using a simple for loop to display the numbers from one to ten is an iterative process.

In a recursive algorithm, current output depends on previous output(s) and input(s) but this is not the case in non-recursive algorithms.

Examples:

(i) Recursive Algorithm

Algorithm F(n)

```
if n = 0 then return 1 // base case
else F(n-1)•n // recursive call
```

(ii) Non-recursive Algorithm

Algorithm Binary(n)

```
count:= 1;
while n > 1 do
count:= count + 1;
n:= ⌊n/2⌋;
end
return count;
```

Compiled: November 2018 Last update: January 2023 Page 37

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

3.1 Analysis of Non-recursive Algorithms

After developing pseudo-code for an algorithm, we wish to analyze its efficiency as a function of the size of the input, n in terms of how many times the elementary operation is performed. Here is a general strategy:

- Decide on parameter n indicating input size
- Identify algorithm's basic operation.
- Check whether the number of time the basic operation is executed depends on some additional property of the input. If so, determine worst, average, and best case for input of size n
- Count the number of operations. Set up a summation corresponding to the number of elementary operations
- Simplify the equation to get as simple of a function f(n) as possible.
- Establish the efficiency class to which the algorithm belongs

NOTE: The core of the algorithm analysis is to find out how the number of the basic operations depends on the size of the input.

Examples of primitive operations:

- o Evaluating an expression e.g. $x^2 + ey$
- o Assigning a value to a variable e.g. $cnt \leftarrow cnt + 1$
- o Indexing into an array e.g. $A[5]$
- o Calling a method e.g. $mySort(A,n)$
- o Returning from a method e.g. $return(cnt)$

In the following search algorithm, there are six basic operations which are marked with a box and labelled 1 – 6.

Compiled: November 2018 Last update: January 2023 Page 38

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Let the $x.length = n$.

Assume a worst case scenario for the above search algorithm:

Number of times operation 1 executes = 1

Number of times operation 2 executes = $n + 1$

Number of times operation 3 executes = n

Number of times operation 4 executes = n

Number of times operation 5 and 6 execute = 1

Total number of operations = $3n + 3$

There are four general rules for counting the operations:

Rule 1: for loops - the size of the loop $\times$ the running time of the body

The running time of a for loop is at most the running time of the statements inside the loop $\times$ the number of iterations.

For ($i = 0$; $i < n$; $i++$)

sum = sum + i;

Example:

a). Find the running time of statements when executed only once:

Compiled: November 2018 Last update: January 2023 Page 39

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The statements in the loop heading have fixed number of operations, hence they have constant running time $O(1)$ when executed only once.

The statement in the loop body has fixed number of operations; hence it has a constant running time when executed only once.

b). Find how many times each statement is executed.

For ($i = 0$; $i < n$; $i++$) // $i = 0$; executed only once: $O(1)$

```
// i < n; n + 1 times O(n)
// i++ n times O(n)
// total time of the loop header: O(1) + O(n) +
O(n) = O(n)
sum = sum + i; // executed n times, O(n)
The loop header plus the loop body will give: O(n) + O(n) = O(n).
```

Loop running time is: $O(n)$

Rule 2: Nested loops - the product of the size of the loops X the running time of the body

The total running time is the running time of the inside statements times the product of the sizes of all the loops

```
sum = 0; for( i = 0; i < n; i++)
for( j = 0; j < n; j++)
sum++;
```

Applying Rule 1 for the nested loop (the 'j' loop) we get $O(n)$ for the body of the outer loop.

The outer loop runs n times, therefore the total time for the nested loops will be

$O(n) * O(n) = O(n*n) = O(n^2)$

What happens if the inner loop does not start from 0?

```
sum = 0; for( i = 0; i < n; i++)
```

Compiled: November 2018 Last update: January 2023 Page 40

CSC315 - Algorithms & Complexity Analysis - Orhionkpaiyo, B. C

```
for( j = i; j < n; j++)
sum++;
```

Here, the number of the times the inner loop is executed depends on the value of i

$i = 0$, inner loop runs n times

$i = 1$, inner loop runs $(n-1)$ times

$i = 2$, inner loop runs $(n-2)$ times

$i = n - 2$, inner loop runs 2 times

$i = n - 1$, inner loop runs once.

Thus we have: $(1 + 2 + \dots + n) = n*(n+1)/2 = O(n^2)$

General rule for nested loops:

Running time is the product of the size of the loops times the running time of the body.

Example:

```
sum = 0;
```

```
for( i = 0; i < n; i++)
for( j = 0; j < 2n; j++)
sum++;
```

We have one operation inside the loops, and the product of the sizes is $2n^2$

Hence the running time is $O(2n^2) = O(n^2)$

Note: if the body contains a function call, its running time has to be taken into consideration.

Compiled: November 2018 Last update: January 2023 Page 41

CSC315 - Algorithms & Complexity Analysis - Orhionkpaiyo, B. C

```
sum = 0;
for( i = 0; i < n; i++)
for( j = 0; j < n; j++)
sum = sum + function(sum);
```

Assume that the running time of `function(sum)` is known to be $\log(n)$.

Then the total running time will be $O(n^2 * \log(n))$

The code to compute the matrix product $C = A * B$ is given below.

```
for (i = 1; i<=n; i++)
for (j = 1; j<=n; j++)
C[i, j] = 0;
for (k = 1; k<=n; k++)
C[i, j] = C[i, j] + A[i, k] * B[k, j];
```

There are 3 nested for loops, each of which runs n times. The innermost loop therefore executes $n*n*n = n^3$ times. The innermost statement, which contains a scalar sum and product takes constant $O(1)$ time. So the algorithm overall takes $O(n^3)$ time.

Rule 3: Consecutive program fragments

The total running time is the maximum of the running time of the individual fragments

```
sum = 0;
for( i = 0; i < n; i++) // runs O(n) time
sum = sum + i;
```

Compiled: November 2018 Last update: January 2023 Page 42

CSC315 - Algorithms & Complexity Analysis - Orhionkpaiyo, B. C

```
sum = 0;
for( i = 0; i < n; i++)
for( j = 0; j < 2*n; j++)
sum++;
```

The first loop runs in $O(n)$ time, the second - $O(n^2)$ time, the maximum is $O(n^2)$

Rule 4: If statement

if C

S1;

else

S2;

The running time is the maximum of the running times of S1 and S2.

3.1.1 General rules for the analysis of programs

In general the running time of a statement or group of statements may be parameterized by the input size and/or by one or more variables. The only permissible parameter for the running time of the whole program is 'n' the input size.

1. The running time of each assignment, read, and write statement can usually be taken to be $O(1)$.
2. The running time of a sequence of statements is determined by the sum rule - i.e. the running time of the sequence is, within a constant factor, the largest running time of any statement in the sequence.

□ Consecutive Statements: S1; S2; S3; ... ; SN, The runtime R of a sequence of statements is the runtime of the statement with the max runtime:

$\max(R(S1), R(S2), \dots, R(SN))$

Compiled: November 2018 Last update: January 2023 Page 43

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

3. The running time of an if-statement is the cost of conditionally executed statements, plus the time for evaluating the condition. The time to evaluate the condition is normally $O(1)$.

The time for an if-then-else construct is the time to evaluate the condition plus the larger of the time needed for the statements executed when the condition is true and the time for the statements executed when the condition is false.

□ Conditional Execution: if (S1) S2; else S3;

The runtime of a conditional execution is

$\max(R(S1), R(S2), R(S3))$

4. The time to execute a loop is the sum, over all times around the loop, the time to execute the body and the time to evaluate the condition for termination (usually the latter is $O(1)$).

Often this time is the product of the number of times around the loop and the largest possible time for one execution of the body, but we must consider each loop separately to be sure.

□ Count inside out: the runtime is the max of the runtime of each statement

multiplied by the total number of times each statement is executed

```
for (i = 0; i < N; i++) {
  for (k = i; k < N; k++) {
    j = p[j]*3;
```

Compiled: November 2018 Last update: January 2023 Page 44

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The table below shows the summary of the rules

Type of code Technique

No loops or $O(1)$

recursion

Loop multiply

Sequence Add

Selection Take the maximum

Method call Apply method's time to size of arguments

We conclude this section with this example:

Algorithm innerProduct

Input: Non-negative integer n , two integer arrays

A and B of size n .

Output: The inner product of the two arrays

```
1. prod ← 0
2 for i ← 0 to n-1 do
3 prod ← prod + A[i]*B[i]
4 return prod
```

- Line 1 is one op (assigning a value).
- Loop initialization is one op (assigning a value).
- Line 3 is five ops per iteration (multiplication, add, 2 array references, assign).
- Line 3 is executed n times; total is $5n$.
- Loop increment is two ops (an addition and an assignment)
- Loop increment is done n times; total is $2n$.
- Loop termination test is one op (a comparison $i < n$) each time.
- Loop termination is done $n+1$ times (n successes, one failure); total is $n+1$.
- Return is one op.

The total is thus: $1 + 1 + 5n + 2n + (n+1) + 1 = 8n + 4$.

Compiled: November 2018 Last update: January 2023 Page 45

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

3.2 Fundamental Computing Algorithms

3.2.1 Search Algorithms

Searching is the algorithmic process of finding a particular item in a collection of items.

- o A search typically answers either True or False as to whether the item is present.
- o On occasion it may be modified to return where the item is found.

i). Linear/Sequential Search Algorithm

The basic idea:

□ Starting at the first element, the algorithm sequentially steps through an array examining each element until it locates the value it is searching for or until the array is exhausted.

The Algorithm

Algorithm linearSearch

Input: An array $A[1..n]$ of n elements and an element x .

Output: j if $x = A[j]$, $1 \leq j \leq n$, and 0 otherwise.

1. $j \leftarrow 1$
2. while $(j < n)$ and $(x \neq A[j])$
3. $j \leftarrow j + 1$
4. end while
5. if $x = A[j]$ then return j else return 0

The algorithm scans the entries in A and compares each entry with x .

- o If after j comparisons, $1 \leq j \leq n$, the search is successful, i.e., $x = A[j]$, j is returned;
- o Otherwise a value of 0 is returned indicating an unsuccessful search.

Compiled: November 2018 Last update: January 2023 Page 46

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The sequential search of a list of integers is illustrated below.

If the item is not in the list, the only way to know it is to compare it against every item present.

- o If there are items, then the sequential search requires comparisons to discover that the item is not there.
- o In the case where the item is in the list, the analysis is not so straightforward.

There are actually three different scenarios that can occur.

- o In the best case we will find the item in the first place we look, at the beginning of the list. We will need only one comparison (i.e. $O(1)$ running time for the best case).
- o In the worst case, we will not discover the item until the very last comparison, the n th comparison.

o Worst still, the item is not in the list

The worst case Analysis of the algorithm

The worst case is when there are no matching elements or the first matching element is the last one on the list. In this case, the algorithm makes the largest number of key comparisons among all possible inputs of size n .

Thus:

$C(n) = n$.

So its worst-case running time is $O(n)$.

Compiled: November 2018 Last update: January 2023 Page 47

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

ii. Binary Search Algorithm

Binary search is a remarkably efficient algorithm for searching in a sorted array. It is a search that is based on divide-and-conquer strategy.

o Although binary search is clearly based on a recursive idea, it can be easily implemented as a non-recursive algorithm, too.

o It works by comparing a search key K with the array's middle element $A[m]$. If they match, the algorithm stops; otherwise, the same operation is repeated recursively for the first half of the array if $K < A[m]$, and for the second half if $K > A[m]$:

As an example, let us apply binary search to searching for $K = 70$ in the array

The iterations of the algorithm are given in the following table:

The following is pseudo code of the non-recursive version.

Compiled: November 2018 Last update: January 2023 Page 48

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

ALGORITHM BinarySearch($A[0..n - 1]$, K)

Input: An array $A[0..n - 1]$ sorted in ascending order and a search key K

Output: An index of the array's element that is equal to K or -1 if there is no such element

$l \leftarrow 0$; $r \leftarrow n - 1$

while $l \leq r$ do

$m \leftarrow \lfloor (l + r)/2 \rfloor$

if $K = A[m]$

return m

else if

$K < A[m]$

$r \leftarrow m - 1$

else

$l \leftarrow m + 1$

return -1

Alternatively, this can be represented as follows:

Low = 0; High = N - 1

while (Low ≤ High)

Mid = (Low + High)/2 //Find the middle index

if (X > A[Mid]) //search second half of array

Low = Mid + 1

else if (X < A[Mid]) // search first half

High = Mid - 1

else return Mid //found X

Compiled: November 2018 Last update: January 2023 Page 49

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Illustration:

Compare X with middle item A[mid], go to left half if X < A[mid] and right half if X > A[mid]. Repeat.

Complexity Analysis of Binary Search

The standard way to analyze the efficiency of binary search is to count the number of times the search key is compared with an element of the array.

o The worst-case inputs include all arrays that do not contain a given search key, as well as some successful searches.

Recall that each comparison eliminates about half of the remaining items from consideration.

o If we start with n items, about n/2 items will be left after the first comparison. After the second comparison, there will be about n/4 then, n/8, n/16, and so on.

o When we split the list enough times, we end up with a list that has just one item. Either that is the item we are looking for or it is not. Either way, we are done.

For an array of size N, it eliminates $\frac{1}{2}$ until 1 element remains.

N, N/2, N/4, N/8, ..., 4, 2, 1

Think of it from the other direction:

- How many times do I have to multiply by 2 to reach N? 1, 2, 4, 8, ..., N/4, N/2, N

- Call this number of multiplications "x".

Compiled: November 2018 Last update: January 2023 Page 50

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

$2^x = N$

$$X = \log_2 N$$

- Binary search is in the logarithmic complexity class.

Detailed Analysis

How many iterations are executed before low > high?

After first iteration: $n/2$ items remaining

2nd iteration: $(n/2)/2 = n/4$

Kth iteration: $n/2^k$ remaining

Worst Case: Last iteration occurs when $n/2^k \geq 1$ and $n/2^{k+1} < 1$ item remaining

i.e. $2^k \leq n$ and $2^{k+1} > n$ [take log of both sides]

Number of iterations is $K \leq \log n$ and $k > \log n - 1$

Worst case running time = $O(\log n)$

3.2.2. Sorting Algorithms

Sorting is ordering a list of objects. We can distinguish two types of sorting.

□ If the number of objects is small enough to fit into the main memory, sorting is called internal sorting.

□ If the number of objects is so large that some of them reside on external storage during the sort, it is called external sorting.

In this section, we will consider the following internal sorting algorithms

□ Bubble sort

□ Insertion sort

□ Selection sort

Compiled: November 2018 Last update: January 2023 Page 51

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

i. Bubble Sort Algorithm

Bubble Sort consists of a simple double for loop.

□ The first iteration of the inner for loop moves through the record array from bottom to top, comparing adjacent keys. If the lower-indexed key's value is greater than its higher-indexed neighbour, then the two values are swapped. Once the smallest value is encountered, this process will cause it to "bubble" up to the top of the array.

□ The second pass through the array repeats this process. However, because we know that the smallest value reached the top of the array on the first pass, there is no need to compare the top two elements on the second pass. Likewise, each succeeding pass through the array compares adjacent elements, looking at one less value than the preceding pass.

The action of the algorithm on the list 42, 20, 17, 13, 28, 14, 23, 15 is illustrated

below.

The above is an illustration of Bubble Sort. Each column shows the array after the iteration with the indicated value of i in the outer for loop. Values above the line in each column have been sorted. Arrows indicate the swaps that take place during a given iteration.

A bubble sort can also be understood if the array to be sorted is envisaged as a vertical column whose smallest elements are at the top and whose largest elements are at the bottom.

Compiled: November 2018 Last update: January 2023 Page 52

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The array is scanned from the bottom up, and two adjacent elements are interchanged if they are found to be out of order with respect to each other.

□ First, items $\text{data}[n-1]$ and $\text{data}[n-2]$ are compared and swapped if they are out of order. Next, $\text{data}[n-2]$ and $\text{data}[n-3]$ are compared, and their order is changed if necessary, and so on up to $\text{data}[1]$ and $\text{data}[0]$. In this way, the smallest element is bubbled up to the top of the array.

□ However, this is only the first pass through the array. The array is scanned again comparing consecutive items and interchanging them when needed, but this time, the last comparison is done for $\text{data}[2]$ and $\text{data}[1]$ because the smallest element is already in its proper position, namely, position 0.

□ The second pass bubbles the second smallest element of the array up to the second position, position 1. The procedure continues until the last pass when only one comparison, $\text{data}[n-1]$ with $\text{data}[n-2]$, and possibly one interchange are performed.

The array [5 2 3 8 1] sorted by bubble sort is shown below.

The sorting activities of the Bubble sort algorithm are summarized as follows:

□ Performs multiple passes over the array

□ In every pass:

Compiled: November 2018 Last update: January 2023 Page 53

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

- o Compare adjacent elements in the list
- o Exchange the elements if they are out of order
- o Each pass moves the largest (or smallest) elements to the end of the array

□ Repeating this process in several passes eventually sorts the array into ascending (or descending) order

The exchange operation, sometimes called a “swap”. Typically, swapping two elements in a list requires a temporary storage location (an additional memory

location). This is illustrated in the diagram below.

The pseudo code of the algorithm is as follows:

1: Version 1

```
bubblesort(data[])
```

```
for i = 0 to data.length-2
```

```
for j = data.length-1 downto i+1
```

```
if elements are out of order swap elements in  
position j and j-1;
```

2: Version 2

```
Bubble Sort (Array A[])
```

```
For i = 1 to n - 1
```

```
For j = 1 to n - i
```

```
If (A[j] > A[j+1]) then
```

```
Swap(A[j], A[j+1])
```

Compiled: November 2018 Last update: January 2023 Page 54

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Analysis of the Bubble Sort Algorithm

The innermost statement, the if, takes $O(1)$ time. It doesn't necessarily take the same time when the condition is true as it does when it is false, but both times are bounded by a constant.

The outer loop executes n times, but the inner loop executes a number of times that depends on i .

The first time the inner for executes, it runs $i = n-1$ times. The second time it runs $n-2$ times as shown in the table below.

The total number of times the inner if statement executes is therefore:

$$(n-1) + (n-2) + \dots + 3 + 2 + 1$$

This is the sum of an arithmetic series.

The value of the sum is $n(n-1)/2$.

So the running time of bubble sort is $O(n(n-1)/2)$, which is

$O((n^2-n)/2)$.

Using the rules for big-O given earlier, this bound simplifies to $O((n^2)/2)$ by ignoring a smaller term, and to $O(n^2)$, by ignoring a constant factor.

Thus, bubble sort is an $O(n^2)$ algorithm.

Compiled: November 2018 Last update: January 2023 Page 55

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

ii) Selection Sort Algorithm

The selection sort improves on the bubble sort by making only one exchange for every pass through the list.

□ In order to do this, a selection sort looks for the largest value as it makes a pass and, after completing the pass, places it in the proper location. As with a bubble sort, after the first pass, the largest item is in the correct place.

□ After the second pass, the next largest is in place. This process continues and requires $n-1$ passes to sort n items, since the final item must be in place after the $(n-1)$ st pass.

The Selection sort performs sorting by searching for the minimum value number and placing it into the first or last position according to the order (ascending or descending). The process of searching the minimum key and placing it in the proper position is continued until the all the elements are placed at right position.

The basic idea of selection sort:

- Consider the positions in the array from left to right
- For each position, find the element that belongs there and put it in place by swapping it with the element that's currently there

The figure shows the entire sorting process. On each pass, the largest remaining item is selected and then placed in its proper location. The first pass places 93, the second pass places 77, the third places 55, and so on.

Compiled: November 2018 Last update: January 2023 Page 56

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The Algorithm

SelectionSort

Input: An array $A[1::n]$ of n elements.

Output: $A[1::n]$ sorted in non-decreasing order.

1. for $i \leftarrow 1$ to $n-1$
2. $k \leftarrow i$
3. for $j \leftarrow i+1$ to n //Find the i th smallest element
4. if $A[j] < A[k]$ then $k \leftarrow j$
5. end for
6. if $k \neq i$ then interchange $A[i]$ and $A[k]$
7. end for

Analysis of Selection sort algorithm

To sort n elements, selection sort performs $n - 1$ passes:

Compiled: November 2018 Last update: January 2023 Page 57

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

- on 1st pass, it performs $n - 1$ comparisons
- on 2nd pass, it performs $n - 2$ comparisons

- on the (n-1)st pass, it performs 1 comparison

Adding up the comparisons for each pass, we get:

$$C(n) = (n - 1) + (n - 2) + \dots + 2 + 1$$

The resulting formula for C(n) is the sum of arithmetic sequence

$$C(n) = (n - 1) + (n - 2) + \dots + 2 + 1 =$$

Formula of the sum of this type of arithmetic sequence is:

Thus, we can simplify our expression for C(n) as follows:

$$C(n) =$$

$$= n^2/2 - n/2$$

In characterizing the time complexity of an algorithm, we'll focus on the largest term in its operation-count expression

$$C(n) = n^2/2 - n/2 \quad n^2/2$$

In addition, we'll typically ignore the coefficient of the largest term

$$\text{Thus } n^2/2 = n^2$$

We specify the largest term using big-O notation.

$$C(n) = n^2/2 - n/2 = O(n^2)$$

Compiled: November 2018 Last update: January 2023 Page 58

CSC315 - Algorithms & Complexity Analysis - Orhionkpaiyo, B. C

Thus the time complexity of selection sort algorithm is $O(n^2)$

That is, selection sort algorithm performs $O(n^2)$ comparisons

We can also compute the number of moves performed by selection sort algorithm.

After each of the n-1 passes to find the smallest remaining element, the algorithm performs a swap to put the element in place.

- n-1 swaps, 3 moves per swap

- $M(n) = 3(n-1) = 3n-3$

Thus selection sort performs $O(n)$ moves.

iii). Insertion Sort Algorithm

Insertion Sort iterates through a list of records. Each record is inserted in turn at the correct position within a sorted list composed of those records already processed.

□ Insertion sort uses two sets of arrays where one stores the sorted data and other on unsorted data.

The primary concept behind insertion sort is to insert each item into its appropriate place in the final list.

□ One way to think about this is to imagine that you have a stack of phone

bills from the past two years and that you wish to organize them by date.

o A fairly natural way to do this might be to look at the first two bills and put them in order. Then take the third bill and put it into the right order with respect to the first two, and so on. As you take

Compiled: November 2018 Last update: January 2023 Page 59

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

each bill, you would add it to the sorted pile that you have already made.

Summary of operations of Insertion Sort

- Take multiple passes over the array
- Keep already sorted array at low-end
- Find next unsorted element
- Insert it in correct place, relative to the ones already sorted

Below is an illustration of Insertion Sort. Each column shows the array after the iteration with the indicated value of i in the outer for loop. Values above the line in each column have been sorted. Arrows indicate the upward motions of records through the array.

The Algorithm

Algorithm insertionSort(A, n)

Input: array A of n integers

Output: the sorted array A

1. for $i \leftarrow 1$ to $n - 1$ do
2. $x \leftarrow A[i]$
3. for $j \leftarrow i - 1$ downto 0 do
4. if $x < A[j]$ then $A[j+1] \leftarrow A[j]$
5. else break
6. $A[j+1] \leftarrow x$
7. return A

Compiled: November 2018 Last update: January 2023 Page 60

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Time complexity Analysis of Insertion Sort

By inspecting the pseudo code, we can determine the maximum number of primitive operations executed by an algorithm, as a function of the input size.

Algorithm insertionSort(A, n) operations total # operations

Input array A of n integers

Output the sorted array A

for $i \leftarrow 1$ to $n - 1$ do 0 0

$x \leftarrow A[i]$ 1 $n - 1$

for $j \leftarrow i - 1$ downto 0 do 0 0

```

if x < A[j] then A[j+1] ← (n -
A[j] 2 1)n
else break 0 0
A[j+1] ← x 1 n - 1
return A 1 1

```

Total $n^2 + n - 1$

There are $n-1$ passes to sort n items. The iteration starts at position 1 and moves through position $n-1$. The maximum number of comparisons for an insertion sort is the sum of the first $n-1$

Total number of comparisons i. e. $C(n) = 1 + 2 + \dots + (n - 1) =$

Compiled: November 2018 Last update: January 2023 Page 61

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

4. Algorithm Design Techniques/Strategies

An algorithm design technique (or “strategy” or “paradigm”) is a general approach to solving problems algorithmically that is applicable to a variety of problems from different areas of computing.

4.1 The Greedy Algorithm Design Technique

The greedy approach is one of the common algorithm design techniques.

□ Greedy algorithms are usually designed to solve optimization problems in which a quantity is to be minimized or maximized.

The greedy method is a simple strategy of progressively building up a solution, one element at a time, by choosing the best possible element at each stage.

o At each stage, a decision is made regarding whether or not a particular input is in an optimal solution. Decisions are never reversed.

This is done by considering the inputs in an order determined by some selection procedure.

□ If the inclusion of the next input, into the partially constructed optimal solution will result in an infeasible solution then this input is not added to the partial solution.

The selection procedure itself is based on some optimization measure. Several optimization measures are plausible for a given problem. Most of them, however, will result in algorithms that generate sub-optimal solutions.

On each step—and this is the central point of this technique—the choice made must be:

- o feasible, i.e., it has to satisfy the problem’s constraints
- o locally optimal, i.e., it has to be the best local choice among all feasible choices available on that step

Compiled: November 2018 Last update: January 2023 Page 62

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

o irrevocable, i.e., once made, it cannot be changed on subsequent steps of the algorithm

For the greedy strategy to work well, it is necessary that the problem under consideration has two characteristics:

□ Greedy choice property: A global optimum can be arrived at by selecting a local optimum

□ Optimal substructure: A problem exhibits optimal substructure if an optimal solution to the problem contains within it optimal solutions to sub-problems.

o i.e. an optimal solution to the problem contains optimal solution to the sub problem

o This implies we can solve sub-problems and build up the solutions to solve larger problems. If an optimal solution A to S begins with activity 1, then $A' = A - \{1\}$ is optimal to $S' = \{i \in S: s_i \neq f_1\}$

To apply the greedy technique to a problem, we must take into consideration the following:

- i. A local criteria to allow the selection,
- ii. A condition to determine if a partial solution can be completed,
- iii. A procedure to test that we have the optimal solution.

4.1.1 Applications of Greedy Algorithms

Greedy algorithms can be applied in wide range of problems which include the following problems:

Compiled: November 2018 Last update: January 2023 Page 63

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

o Activity selection problem

o Minimum spanning tree

o Knapsack problem

o Unit task scheduling with deadline

However, its applications are not limited to these problem areas.

1. Activity Selection Problem

Problem: Schedule an exclusive resource in competition with other entities.

Examples include:

- o Scheduling a hall for lectures (only one course can hold at a time) when several groups want to use it.
- o Renting out some piece of equipment to different people is another

example.

o Others include requests for boats to use a repair facility while they are in port, and planning weekend schedule.

The activity-selection problem is the problem of selecting the largest set of mutually compatible activities i.e. finding a maximum set of non-overlapping tasks (each with a fixed start and finish time) that can be completed by a single processor

Suppose we are given a set of n lectures, and lecture i starts at time s_i and ends at time f_i . The goal is to use the minimum number classrooms to schedule all lectures so that no two occur at the same time in the same room. As an illustration of the problem, consider the sample instance

Compiled: November 2018 Last update: January 2023 Page 64

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

(a) An instance of the lectures (a through j).

(b) A solution in which all lectures are scheduled using 3 classrooms: each row represents a set of lectures that can all be scheduled in a single classroom.

The diagram below helps to visualize this (b) by incorporating time.

Problem Description: There is a set of activities $S = \{a_1, a_2, \dots, a_n\}$. Each activity a_i has a start time s_i and a finish time f_i , where $f_i > s_i \geq 0$. The duration of activity a_i is (s_i, f_i) . Two activities a_i and a_j are compatible if $s_i \geq f_j$ or $s_j \geq f_i$. Two activities a_i and a_j overlap if $[s_i, f_i] \cap [s_j, f_j] \neq \emptyset$. The objective is to select a maximum number of pair wise non-overlapping activities (i.e. select a subset of S that has maximum number of activities that are mutually compatible).

Compiled: November 2018 Last update: January 2023 Page 65

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Consider a set of requests for a room. Only one person can reserve the room at a time, and you want to allow the maximum number of requests.

The requests for periods (s_i, f_i) are:

$(1,4), (3,5), (0,6), (5,7), (3,8), (5,9), (6,10), (8,11), (8,12), (2,13), (12,14)$

Which ones should we schedule?

Different approaches for selecting activities/jobs:

- i. Earliest start time: Considers jobs in ascending order of start times s_i
- ii. Shortest request time: Considers jobs in ascending order of interval length $f_i - s_i$
- iii. Fewest conflicting jobs: For each job, count the number of conflicting jobs c_i . Schedule in ascending order of conflicts c_i .
- iv. Earliest finish time: Considers jobs in ascending order of start time f_i .

This is mostly used because it yields optimal choice. It leaves the resource available for as many other activities as possible.

Example:

i a1 a2 a3 a4 a5 a6 a7 a8 a9 a10 a11

si 1 3 0 5 3 5 6 8 8 2 12

fi 4 5 6 8 8 9 10 11 12 13 14

Possible sets of mutually compatible activities:

i. {a3, a9, a11}

ii. {a1, a4, a8, a11}

iii. {a2, a4, a9, a11}

Observe that ii and iii have the largest subsets of compatible activities

Exercise: Using the above table, determine the possible sets of mutually compatible activities by applying the first three approaches above.

Compiled: November 2018 Last update: January 2023 Page 66

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Points to remember

- For this algorithm we have a list of activities with their starting time and finishing time.
- Our goal is to select maximum number of non-conflicting activities that can be performed by a person or a machine, assuming that the person or machine involved can work on a single activity at a time.
- Any two activities are said to be non-conflicting if starting time of one activity is greater than or equal to the finishing time of the other activity.
- In order to solve this problem we first sort the activities as per their finishing time in ascending order.
- Then we select non-conflicting activities.

Problem

Consider the following 8 activities with their starting and finishing time.

Activity a1 a2 a3 a4 a5 a6 a7 a8

start 1 0 1 4 2 5 3 4

finish 3 4 2 6 9 8 5 5

Our goal is to find non-conflicting activities.

For this we follow the given steps

1. sort the activities as per finishing time in ascending order
 2. select the first activity
 3. select the new activity if its starting time is greater than or equal to the previously selected activity
- REPEAT step 3 till all activities are checked

Step 1: sort the activities as per finishing time in ascending order

Compiled: November 2018 Last update: January 2023 Page 67

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Sorted Activity a3 a1 a2 a7 a8 a4 a6 a5

start 1 1 0 3 4 4 5 2

finish 2 3 4 5 5 6 8 9

Step 2: select the first activity

Step 3: select next activity whose start time is greater than or equal to the finish time of the previously selected activity

Pseudo Code

Set $i = 0$; //pointing at first element

for $j = 1$ to $n-1$ do

if start time of $j \geq$ finish time of i then

Print j

Set $i = j$

endif

endfor

The Greedy Activity Selector Algorithm:

Greedy_activity_selector (s, f)

1. Sort input activities in order of increasing finishing time ($f_1 \leq f_2 \leq \dots \leq f_n$)

2. $n \leftarrow \text{length}[s]$

3. $A \leftarrow \{i\}$

4. $j \leftarrow 1$

5. for $i \leftarrow 2$ to n

6. if $s_i \geq f_j$ then

7. $A \leftarrow A \cup \{i\}$

8. $j \leftarrow i$

9. return A

Compiled: November 2018 Last update: January 2023 Page 68

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The set A collects the selected activities. The variable j specifies the most recent addition to A . Since the activities are considered in order of non-decreasing finishing time, f_j is always the maximum finishing time of any activity in A .

Analysis of the Algorithm

Running time of line 1 in the above depends on the sort algorithm used.

With merge sort algorithm, it requires $O(n \log n)$ time.

$O(n)$ for the greedy collection of activities (i.e The algorithm takes as input a set of activities S (each activity stored as a pair of numbers (s_i ,

fi)) and the total number of activities n).

Total running time is $O(n \log n) + O(n) = O(n \log n)$

2. Minimum Spanning Tree

A spanning tree of a graph $G = (V, E)$ is a tree that contains all vertices of V and is a sub-graph of G . A single graph can have multiple spanning trees.

A Tree is a connected undirected graph that contains no cycles.

A Cycle is a sequence of nodes and edges without repetitions other than the starting and ending nodes. Example

Compiled: November 2018 Last update: January 2023 Page 69

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

An Undirected Graph is a graph where the edges have no directions. An undirected graph is connected when there is a path between every pair of vertices. In a connected graph, there are no unreachable vertices

2 2

1 1

3 3

(a) Undirected graph (b) Directed graph

Lemma 1: Let T be a spanning tree of a graph G . Then

1. Any two vertices in T are connected by a unique simple path.
2. If any edge is removed from T , then T becomes disconnected.
3. If we add any edge into T , then the new graph will contain a cycle.
4. Number of edges in T is $n-1$.

A spanning tree for a connected graph is a tree whose vertex set is the same as the vertex set of the given graph, and whose edge set is a subset of the edge set of the given graph. i.e., any connected graph will have a spanning tree.

Weight of a spanning tree $w(T)$ is the sum of weights of all edges in T . The Minimum spanning tree (MST) is a spanning tree with the smallest possible weight.

The following diagrams illustrate a graph and the possible spanning trees from the graph.

Compiled: November 2018 Last update: January 2023 Page 70

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

A greedy method to obtain the minimum spanning tree would construct the tree edge by edge, where each edge is chosen accounting to some optimization criterion. An obvious criterion would be to choose an edge which adds a minimum weight to the total weight of the edges selected so far. There are two ways in which this criterion can be achieved.

- i. The set of edges selected so far always forms a tree, the next edge to be

added is such that not only it adds a minimum weight, but also forms a tree with the previous edges; it can be shown that the algorithm results in a minimum cost tree; this algorithm is called Prim's algorithm.

ii. The edges are considered in non decreasing order of weight; the set T of edges at each stage is such that it is possible to complete T into a tree; thus T may not be a tree at all stages of the algorithm; this also results in a minimum cost tree; this algorithm is called Kruskal's algorithm.

i). Prim's algorithm.

The algorithm grows the spanning tree starting from an arbitrary vertex. Let $G = (V, E)$, where for simplicity V is taken to be the set of integers $\{1, 2, \dots, n\}$. The algorithm begins by creating two sets of vertices: $X = \{1\}$ and $Y = \{2, 3, \dots, n\}$. It then grows a spanning tree, one edge at a time. At each step, it finds an edge $(x,$

Compiled: November 2018 Last update: January 2023 Page 71

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

$y)$ of minimum weight, where $x \in X$ and $y \in Y$ and moves y from Y to X . This edge is added to the current minimum spanning tree edges in T . This step is repeated until Y becomes empty.

The main idea of Prim's algorithm:

- Start with one (any) vertex.
- Branch outwards to grow your connected component.
- Consider only edges that leave the connected component.
- Add smallest considered edge to your connected component.
- Continue until a spanning tree is created

Steps for finding MST using Prim's Algorithm:

1. Create MST set that keeps track of vertices already included in MST.
2. Assign key values to all vertices in the input graph. Initialize all key values as INFINITE (∞). Assign key values like 0 for the first vertex so that it is picked first.
3. While MST set doesn't include all vertices.
 1. Pick vertex u which is not in MST set and has minimum key value. Include ' u ' to MST set.
 2. Update the key value of all adjacent vertices of u . To update, iterate through all adjacent vertices. For every adjacent vertex v , if the weight of edge $u.v$ less than the previous key value of v , update key value as a weight of $u.v$.

Simply put, the implementation of Prim's Algorithm is explained in the following steps-

Step1:

- Randomly choose any vertex.

□ The vertex connecting to the edge having least weight is usually selected.

Compiled: November 2018 Last update: January 2023 Page 72

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Step2:

□ Find all the edges that connect the tree to new vertices.

□ Find the least weight edge among those edges and include it in the existing tree.

□ If including that edge creates a cycle, then reject that edge and look for the next least weight edge.

□ Keep repeating this step 2 until all the vertices is included and Minimum Spanning Tree (MST) is obtained.

The following diagrams illustrate the operation of Prim's algorithm

Step 1: Select edge 1- 6

Compiled: November 2018 Last update: January 2023 Page 73

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Step 2: Add edge 6 - 5

Step 3: Add edge 5 - 4

Step 4: Add edge 4 - 3

Step 5: Add edge 3 - 2

Compiled: November 2018 Last update: January 2023 Page 74

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Step 6: Add edge 2 - 7

The cost of the MST is as shown in the following table

Selected Edges Weight

1-6 10

6-5 25

5-4 22

4-3 12

3-2 16

2-7 14

Total Weight 99

Consider another example

Compiled: November 2018 Last update: January 2023 Page 75

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The final generated MST is as shown below

Selected Edges Weight

1-7 1

7-2 4

7-3 9

3-4 3

4-5 17

1-6 23

Total Weight 57

Compiled: November 2018 Last update: January 2023 Page 76

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The Algorithm

$\{N[y]$ is neighbour of $y\}$

$\{C[y]$ is cost of edge connecting $y\}$

$\{\text{Vertex } y \text{ not adjacent to } 1\}$

Time complexity Analysis of the Algorithm

The time complexity of the algorithm is computed as follows:

The for loop in Step 2 requires $O(n)$ time.

The time taken by steps 3 - 6 is $O(n)$.

The time taken by Step 9 to search for a vertex y closest to X is $O(n)$ per iteration. This is because the algorithm inspects each entry in the vector representing the set Y . Since this step is executed $n - 1$ times, the overall time taken by Step 9 is $O(n^2)$.

Steps 10 - 13 cost $O(1)$ time per iteration for a total of $O(n)$ time.

The for loop in Step 14 is executed $2m$ times, where $m = |E|$. This is because each edge (y, w) is inspected twice: once when y is moved to X and the other when w is moved to X . Hence, the overall time required by Step 14 is $O(m)$.

The test in Step 15 is executed exactly m times, and Steps 16 and 17 are executed at most m times. Thus, Steps 14 to 17 cost $O(m)$ time.

Compiled: November 2018 Last update: January 2023 Page 77

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

It follows that the time complexity of the algorithm is $O(m + n^2) = O(n^2)$.

However, using a heap improves the algorithm to $O(m \log n)$ time

ii). Kruskal's Algorithm

Kruskal's algorithm works by maintaining a forest consisting of several spanning trees that are gradually merged until finally the forest consists of exactly one tree: a minimum cost spanning tree.

Kruskal's algorithm is summarized as follows:

- Sort the edges in graph G by non-decreasing weight.
- For each edge in the sorted list, include that edge in the spanning tree T if it does not form a cycle with the edges currently included

in T ; otherwise discard it.

Consider the weighted graph below. The first edge that is added is $(1, 2)$, followed by edges $(1, 3)$, $(4, 6)$, and $(5, 6)$. Notice that edge $(2, 3)$ creates a cycle as shown in figure f, and hence is discarded. For the same reason edge $(4, 5)$ is also discarded, as shown in figure g. Finally, edge $(3, 4)$ is included, which results in the minimum spanning tree (V, T) shown in fig. (h).

Compiled: November 2018 Last update: January 2023 Page 78

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The Algorithm

Time complexity

We analyze the time complexity of the algorithm as follows:

Steps 1 and 2 cost $O(m \log m)$ and $\Theta(n)$, respectively, where $m = |E|$.

Step 6 costs $\Theta(1)$, and since it is executed at most m times, its total cost is $O(m)$.

Step 7 is executed exactly $n - 1$ times for a total of $\Theta(n)$ time.

The union operation is executed $n - 1$ times

The find operation is executed at most $2m$ times.

The overall cost of these two operations (i.e. union and find) is $O(m \log^* n)$.

Compiled: November 2018 Last update: January 2023 Page 79

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Thus, the overall running time of the algorithm is dominated by the sorting step, i.e., $O(m \log m)$.

iii). The Knapsack Problem

There are two variants of the knapsack problem:

- Integral or 0-1 Knapsack problem,
- Fractional knapsack problem

The 0-1 knapsack problem is posed as follows. A thief robbing a store finds n items; the i th item is worth v_i dollars and weighs w_i pounds, where v_i and w_i are integers. He wants to take as valuable a load as possible, but he can carry at most W pounds in his knapsack for some integer W . Which items should he take? (This is called the 0-1 knapsack problem because each item must either be taken or left behind; the thief cannot take a fractional amount of an item or take an item more than once e.g. TV set)

In the fractional knapsack problem, the setup is the same, but the thief can take fractions of items, rather than having to make a binary (0-1) choice for each item. You can think of an item in the 0-1 knapsack problem as being like a gold ingot (mass of metal), while an item in the fractional knapsack problem is more like gold dust.

Both knapsack problems exhibit the optimal-substructure property.

o For the 0-1 problem, consider the most valuable load that weighs at most W pounds. If we remove item j from this load, the remaining load must be the most valuable load weighing at most $W - w_j$ that the thief can take from the $n - 1$ original items excluding j .

Compiled: November 2018 Last update: January 2023 Page 80

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

o For the comparable fractional problem, consider that if we remove a weight w of one item j from the optimal load, the remaining load must be the most valuable load weighing at most $W - w$ that the thief can take from the $n - 1$ original items plus $w_j - w$ pounds of item j .

However, only the fractional knapsack problem has the greedy choice property. Thus, the fractional knapsack problem is solvable by a greedy strategy, whereas the 0-1 problem is not (but solvable by dynamic programming). Therefore, our focus is the fractional knapsack problem.

o The Fractional Knapsack Problem

The fractional knapsack problem is defined as follows:

Given a list of n objects say $\{I_1, I_2, \dots, I_n\}$ and a knapsack (or bag) with capacity, M . Each object I_i has a weight W_i and a profit of P_i . If a fraction x_i (where $x_i \in \{0, \dots, 1\}$) of an object I_i is placed into a knapsack, then a profit of $p_i x_i$ is earned.

The problem (or objective) is to fill a knapsack up to its maximum capacity M , which maximizes the total profit earned.

Mathematically:

Where p_i and w_i are the profit and weight of i th object and x_i is the fraction of i th object to be selected.

Compiled: November 2018 Last update: January 2023 Page 81

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Example:

Given $n = 3$, $(p_1, p_2, p_3) = \{25, 24, 15\}$

$(w_1, w_2, w_3) = \{18, 15, 10\}$ $M = 20$

Solution

Some of the feasible solutions are shown in the following table.

Solution No x_1 x_2 x_3 $\sum w_i x_i$ $\sum p_i x_i$

1 1 2/15 0 20 28.2

2 0 2/3 1 20 31.0

3 0 1 1/2 20 31.5

These solutions are obtained by different greedy strategies.

Greedy strategy I: In this case, the items are arranged by their profit values.

Here the item with maximum profit is selected first. If the weight of the object is less than the remaining capacity of the knapsack then the object is selected full and the profit associated with the object is added to the total profit.

Otherwise, a fraction of the object is selected so that the knapsack can be filled exactly. This process continues from selecting the highest profitable object to the lowest profitable object till the knapsack is exactly full.

Thus, the first item has the maximum profit (25) and is selected first. After filling the item ($w = 18$), the sack is left with only 2 capacity (i.e. $20 - 18$).

Next, a fraction of the second item (profit = 24) is selected. Since the weight, 15, exceeds the maximum, a fraction of it, $\frac{2}{15}$, is taken. Now the knapsack is full (i.e. 20) so 3rd item is not selected. Hence, the total profit = 28.2 units and the solution set is $(x_1, x_2, x_3) = (1, \frac{2}{15}, 0)$.

Compiled: November 2018 Last update: January 2023 Page 82

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Greedy strategy II: In this case, the items are arranged by fair weights. Here the item with minimum weight is selected first and the process continues like greedy strategy-I till the knapsack is exactly full.

Thus, 3rd, and 2nd items are selected. After selecting the 3rd item, the sack remains 10 to be filled, so only a fraction ($\frac{10}{20}$) of the 2nd item is taken. That is of $15 = 10$. Hence, the profit is 31.0

Greedy strategy III: In this case, the items are arranged by profit/weight ratio and the item with maximum profit/weight ratio is selected first and the process continues like greedy strategy-I till the knapsack is exactly full.

Using this approach, select those objects with maximum value of $\frac{p_i}{w_i}$, that is,

select those objects first with maximum profit per unit weight. Since $\frac{p_i}{w_i} = 1.3, 1.6, 1.5$ respectively, select 2nd object first, then 3rd object followed by 1st object.

The total profit (i.e. 31.5 (i.e. $24 + 7.5$)) and the solution set

$(x_1, x_2, x_3) = (0, 1, \frac{5}{15})$ i.e. total weight selected from $x_1, x_2, x_3 = 0, 15, 5 = 20$.

Therefore, it is clear from the above strategies that the Greedy method generates optimal solution if we select the objects with respect to their profit to weight ratios.

Compiled: November 2018 Last update: January 2023 Page 83

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The Algorithm

Time Complexity Analysis

Sorting of n items in decreasing order of the ratio takes $O(n \log n)$

time (which is the most efficient runtime possible for sort algorithms

Line 6 (i.e. the while loop) takes $O(n)$ time

Therefore, total time is $O(n \log n)$

iv). Unit Task Scheduling with Deadlines

A unit-time task is a job, such as a program to be run on a computer that requires exactly one unit of time to complete. Given a finite set S of unit-time tasks, a schedule for S is a permutation of S specifying the order in which to perform these tasks.

o The first task in the schedule begins at time 0 and finishes at time 1, the second task begins at time 1 and finishes at time 2, and so on.

Compiled: November 2018 Last update: January 2023 Page 84

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

o Because each job takes the same amount of time, we will think of a Schedule S as consisting of a sequence of job “slots” $1, 2, 3, \dots$ where $S(t)$ is the job scheduled in slot, t

The problem of scheduling unit-time tasks with deadlines and penalties for a single processor has the following inputs:

- a set $S = \{a_1, a_2, \dots, a_n\}$ of n unit-time tasks;
- a set of n integer deadlines $d_1, d_2, \dots, d_n$, such that each d_i satisfies $1 \leq d_i \leq n$ and task a_i is supposed to finish by time d_i ; and
- a set of n nonnegative weights or penalties $w_1, w_2, \dots, w_n$, such that we incur a penalty of w_i if task a_i is not finished by time d_i , and we incur no penalty if a task finishes by its deadline.

We wish to find a schedule for S that minimizes the total penalty incurred for missed deadlines.

Task Scheduling Problem:

Given is a set S of n tasks $\{a_1, \dots, a_n\}$ and a single processor. Each task takes a unit time to execute and tasks have deadlines $d_1, \dots, d_n$. Tasks have non-negative penalties $w_1, \dots, w_n$ if miss the deadline.

Goal: Find a schedule S where the total penalty of the tasks that miss their deadline is minimized – i.e., the total penalty of the tasks that meet their deadline is maximized

NOTE:

o A feasible solution is a subset of jobs J such that each job is completed by its deadline.

o An optimal solution is a feasible solution with maximum profit value.

Compiled: November 2018 Last update: January 2023 Page 85

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

A Greedy Strategy for Unit Task Scheduling:

- i. Sort all the given jobs in decreasing order of their profit.
- ii. Check the value of maximum deadline and draw a Gantt chart where maximum time on Gantt chart is the value of maximum deadline.
- iii. Pick up the jobs one by one on the Gantt chart as far as possible from 0 ensuring that the job gets completed before its deadline.

Given the following six jobs, their deadlines and associated profits, we illustrate the above strategy as follows:

Jobs J1 J2 J3 J4 J5 J6

Deadlines 5 3 3 2 4 2

Profits 20 18 19 30 12 10

Step 1: Sort all the given jobs in decreasing order of their profit

Jobs J4 J1 J3 J2 J5 J6

Deadlines 2 5 3 3 4 2

Profits 30 20 19 18 12 10

Step 2:

Value of maximum deadline = 5.

So, draw a Gantt chart with maximum time on Gantt chart = 5 units as shown

Gantt chart

Compiled: November 2018 Last update: January 2023 Page 86

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Step 3:

1. First, take job J4 since its deadline is 2, so we place it in the first empty cell before deadline 2
2. Then take job J1 since its deadline is 5, so we place it in the first empty cell before deadline 5 as-
3. Next, take job J3 since its deadline is 3, so we place it in the first empty cell before deadline 3
4. Next, take job J2. Since its deadline is 3, so we place it in the first empty cell before deadline 3. But the second and third cells are already filled, so we place job J2 in the first cell
5. Next, take job J5. Since its deadline is 4, so we place it in the first empty cell before deadline 4

NOTE: The only job left is J6 whose deadline is 2. All the slots before deadline 2 are already occupied therefore J6 cannot be completed.

Thus the optimal schedule of the jobs is: J2, J4, J3, J5, J1

Total profit: $30 + 20 + 19 + 18 + 12 = 99$

Consider another example:

Using greedy algorithm, find an optimal schedule for the following jobs with $n = 6$.

Profits: $(P_1, P_2, P_3, P_4, P_5, P_6) = (20, 15, 10, 7, 5, 3)$ and

Deadline $(d_1, d_2, d_3, d_4, d_5, d_6) = (3, 1, 1, 3, 1, 3)$

Compiled: November 2018 Last update: January 2023 Page 87

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Solution:

Job i 1 2 3 4 5 6

Profit g_i 20 15 10 7 5 3

Deadline d_i 3 1 1 3 1 3

Steps:

i. Here jobs are already sorted in decreasing order of their profit

Let $P = \min(6, 3) = 3$

$F = 0 \ 1 \ 2 \ 3$

Job 0 0 0 0

selected

ii. $d_1 = 3$: assign job 1 to position 3

$F = 0 \ 1 \ 2 \ 0$

Job 0 0 0 1

selected

iii. $d_2 = 1$: assign job 2 to position 1

$F = 0 \ 0 \ 2 \ 0$

Job 0 2 0 1

selected

iv. $d_3 = 1$: No free position, so reject the job

v. $d_4 = 3$: assign job 4 to position 2 as position 3 is not free, but position 2 is free

$F = 0 \ 0 \ 0 \ 0$

Job 0 2 4 1

selected

Compiled: November 2018 Last update: January 2023 Page 88

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

vi. Now, no more free position is left so no more jobs can be scheduled

The final optimal sequence: Execute the job order 2, 4, 1 with total profit value 42

The Algorithm

1:

2:

Sort all the n jobs in decreasing order of their profit

for $i = 1$ to n do

Set $k = \min(d_{\max}, \text{DEADLINE}(i))$ //where

$\text{DEADLINE}(i)$ denotes

deadline of i th job

while $k \geq 1$ do

if timeslot[k] is EMPTY then

timeslot[k] = job(i)

break

endif

Set $k = k - 1$

endwhile

endfor

Compiled: November 2018 Last update: January 2023 Page 89

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Analysis of the algorithm

Sort task according to decreasing order of deadline = $O(n \log n)$

for each task find slot in array of size $n = O(n^2)$

Total time = $O(n \log n) + O(n^2) = O(n^2)$

Compiled: November 2018 Last update: January 2023 Page 90

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

5. Divide-and-Conquer Algorithm Design Strategy

Many useful algorithms are recursive in structure: to solve a given problem, they call themselves recursively one or more times to deal with closely related sub-problems.

These algorithms typically follow a divide-and-conquer approach:

- o they break the problem into several sub-problems that are similar to the original problem but smaller in size,
- o solve the sub-problems recursively, and
- o then combine these solutions to create a solution to the original problem.

The divide-and-conquer paradigm involves three steps at each level of the recursion:

Divide the problem into a number of sub-problems.

Conquer the sub-problems by solving them recursively. If the sub-problem sizes are small enough, however, just solve the sub-problems in a straightforward manner.

Combine the solutions to the sub-problems into the solution for the original

problem.

When an algorithm contains a recursive call to itself, its running time can often be described by a recurrence. Recurrence is briefly explained in the following section.

5.1 Recurrences

A recurrence is a recursive description of a function, or in other words, a description of a function in terms of itself.

- o Each term of the sequence is defined as a function of the preceding terms.

Compiled: November 2018 Last update: January 2023 Page 91

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Like all recursive structures, a recurrence consists of one or more base cases and one or more recursive cases. Each of these cases is an equation or inequality, with some function value $f(n)$ on the left side.

- o The base cases give explicit values for a (typically finite, typically small) subset of the possible values of n .
- o The recursive cases relate the function value $f(n)$ to function value $f(k)$ for one or more integers $k < n$;
- o typically, each recursive case applies to an infinite number of possible values of n .

For example, the following recurrence (written in two different but standard ways) describes the identity function $f(n) = n$:

In both presentations, the first line is the only base case, and the second line is the only recursive case.

Recurrences arise naturally in the analysis of algorithms, especially recursive algorithms. In many cases, we can express the running time of an algorithm as a recurrence, where the recursive cases of the recurrence correspond exactly to the recursive cases of the algorithm.

The recurrence: $T(n) = aT(n/b) + f(n)$

is called the general divide-and-conquer recurrence and describes the running time of an algorithm that divides a problem of size n into a sub-problems, each of size n/b , where a and b are positive constants. The ' a ' sub-problems are solved recursively, each in $T(n/b)$.

- o The function encompasses the cost of dividing the problem and combining the results of the sub-problems.

Compiled: November 2018 Last update: January 2023 Page 92

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The Master Theorem:

The Master theorem provides a “cookbook” method for solving recurrences. It may be applied whenever appropriate, rather than re-deriving the solution for the recurrence. To use the master method, we simply determine which case (if any) of the master theorem applies

For any recurrence relation of the form

$$T(n) = aT(n/b) + cn^k, T(1) = c,$$

The following relationship may hold:

Where $a, b \geq 1$, $k \geq 0$, and $c > 0$ are constants

The above relation also appear in some text as:

$$f(n) = af(n/b) + cn^k$$

We can apply the theorem to solve problems as follows

Example 1:

$$T(n) = 3T(n/5) + 8n^2$$

Because $a = 3$, $b = 5$, $c = 8$, and $k = 2$, we find that $3 < 5^2$.

Applying case (3) of the theorem, $T(n) = \Theta(n^2)$.

Example 2:

$$T(n) = 2T(n/2) + n, T(1) = 1$$

Because $a = 2$, $b = 2$, $c = 1$, and $k = 1$, we find that $2 = 2^1$.

Applying case (2) of the theorem, $T(n) = \Theta(n \log n)$.

Compiled: November 2018 Last update: January 2023 Page 93

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

5.2 Quicksort Algorithm

The quicksort algorithm is a sorting algorithm that sorts a collection by choosing a pivot point, and partitioning the collection around the pivot, so that elements smaller than the pivot are before it, and elements larger than the pivot are after it.

ALGORITHM Quicksort($A[l..r]$)

//Sorts a sub-array by quicksort

Input: Sub-array of array $A[0..n - 1]$, defined by its left and right indices l and r

Output: Sub-array $A[l..r]$ sorted in nondecreasing order

if $l < r$

$s \leftarrow \text{Partition}(A[l..r])$ // s is a split position

Quicksort($A[l..s - 1]$)

Quicksort($A[s + 1..r]$)

Partition: The performance of quicksort depends heavily on the performance of the split operation. The effect of splitting from l to r is:

- $x = A[l]$ is moved to its correct location at $A[m]$;
- no item in $A[l..m-1]$ is larger than x ;

- no item in $A[m+1..r]$ is smaller than x .

ALGORITHM HOARE-PARTITION(A, p, r)

```

1.  $x = A[p]$ 
2.  $i = p - 1$ 
3.  $j = r + 1$ 
4. while true
5. repeat
6.  $j = j - 1$ 
7. until  $A[j] \leq x$ 
8. repeat
9.  $i = i + 1$ 

```

Compiled: November 2018 Last update: January 2023 Page 94

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

```

10. until  $A[i] \geq x$ 
11. if  $i < j$ 
12. exchange  $A[i]$  with  $A[j]$ 
13. else return  $j$ 

```

The left-to-right scan, denoted below by index pointer i , starts with the second element. Since we want elements smaller than the pivot to be in the left part of the sub-array, this scan skips over elements that are smaller than the pivot and stops upon encountering the first element greater than or equal to the pivot.

The right-to-left scan, denoted below by index pointer j , starts with the last element of the sub-array. Since we want elements larger than the pivot to be in the right part of the sub-array, this scan skips over elements that are larger than the pivot and stops on encountering the first element smaller than or equal to the pivot.

After both scans stop, three situations may arise, depending on whether or not the scanning indices have crossed. If scanning indices i and j have not crossed, i.e. $i < j$, we simply exchange $A[i]$ and $A[j]$ and resume the scans by incrementing i and decrementing j , respectively:

If the scanning indices have crossed over, i.e., $i > j$, we will have partitioned the sub-array after exchanging the pivot with $A[j]$:

Finally, if the scanning indices stop while pointing to the same element, i.e., $i = j$, the value they are pointing to must be equal to p . Thus, we have the sub-array partitioned, with the split position $s = i = j$:

Compiled: November 2018 Last update: January 2023 Page 95

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

We can combine the last case with the case of crossed-over indices ($i > j$) by exchanging the pivot with $A[j]$ whenever $i \geq j$.

Run Time Analysis of Quicksort

i. Worst-case partitioning

The running time of QUICKSORT is dominated by the time spent in the partition procedure. Each time the PARTITION procedure is called, a pivot element is selected, and this element is never included in any future recursive calls to QUICK-SORT and PARTITION.

Thus, there can be at most n calls to PARTITION over the entire execution of the quicksort algorithm.

One call to PARTITION takes $O(1)$ time plus an amount of time that is proportional to the number of iterations of the loop.

The partitioning costs $2(n)$ time. Since the recursive call on an array of size 0 just returns,

$T(0) = 2(1)$, and the recurrence for the running time is

$$\begin{aligned} T(n) &= T(n-1) + T(0) + 2(n) \\ &= T(n-1) + 2(n) \end{aligned}$$

Intuitively, if we sum the costs incurred at each level of the recursion, we get an arithmetic series (which evaluates to $2(n^2)$). Thus, if the partitioning is maximally unbalanced at every recursive level of the algorithm, the running time is $2(n^2)$ i.e. $O(n^2)$

Compiled: November 2018 Last update: January 2023 Page 96

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Therefore the worst-case running time of quicksort is $2(n^2)$, which occurs when the input array is already completely sorted

ii. Best-case partitioning

In the most even possible split, PARTITION produces two sub-problems, each of size no more than $n/2$, since one is of size $\lfloor n/2 \rfloor$ and one of size $\lfloor n/2 \rfloor - 1$. In this case, quicksort runs much faster. The recurrence for the running time is then

$$T(n) \leq 2T(n/2) + 2(n),$$

which has the solution $T(n) = O(n \lg n)$

Thus, the equal balancing of the two sides of the partition at every level of the recursion produces an asymptotically faster algorithm.

Compiled: November 2018 Last update: January 2023 Page 97

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

5.3 Binary Search Tree

A Binary Search Tree (BST), also called ordered binary tree, has the following property:

For each node n of the tree,

- o all values stored in its left sub-tree (the tree whose root is the left child) are less than value v stored in n , and
- o all values stored in the right sub-tree are greater than v .

NOTE: Storing multiple copies of the same value in the same tree is avoided. An attempt to do so can be treated as an error.

The meaning of “less than” or “greater than” depend on the type of values stored in the tree: It is “ $<$ ” and “ $>$ ” for numerical values and alphabetical order in the case of strings.

Below is a representation of binary search tree.

The keys in a binary search tree are always stored in such a way as to satisfy the binary-search-tree property:

Let x be a node in a binary search tree.

- o If y is a node in the left sub-tree of x , then $y.key \leq x.key$.
- o If y is a node in the right sub-tree of x , then $y.key \geq x.key$.

Compiled: November 2018 Last update: January 2023 Page 98

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

For each node x it encounters, it compares the key k with $x.key$.

- o If the two keys are equal, the search terminates.
- o If k is smaller than $x.key$, the search continues in the left sub-tree of x , since the binary-search tree property implies that k could not be stored in the right sub-tree.
- o Symmetrically, if k is larger than $x.key$, the search continues in the right sub-tree.

The nodes encountered during the recursion form a simple path downward from the root of the tree, and thus the running time of TREE-SEARCH is $O(h)$, where h is the height of the tree.

□ The height of a tree is the number of nodes in the longest path from the root node to a leaf node.

□ In other words, if the depth of a binary search tree is n nodes so the running time of the algorithm will be $O(n)$.

However, for balanced binary tree, it is logarithmic running time. A tree is balanced if, for each node, the node’s sub-trees have the same height or have heights that differ by 1. For a balanced tree with n nodes, the height is $O(\log_2 n)$ which gives a worst-case time complexity that is logarithmic ($O(\log_2 n)$)

Exercises

1. Differentiate between Binary Search and Binary Search Tree

2. Solve the following recurrences using the master theorem

(a). $T(n) = 3T(n/4) + n$.

(b). $T(n) = T(n/2) + c$

(c). $T(n) = T(n-1) + 1$

Compiled: November 2018 Last update: January 2023 Page 99

Paper 5 of 5 — CSC 315 MATERIAL

Level 300 • First Semester • Handout

Department of Computer Science

Federal University of Petroleum Resources, Effurun,

Delta State

CSC315

ALGORITHMS & COMPLEXITY ANALYSIS

[Orhionkpaiyo, B. C]

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Course Aim:

Algorithms (along with data structures) are the fundamental “building blocks” from which programs are constructed. Only by fully understanding them is it possible to write very effective programs. An algorithmic solution to a computational problem will usually involve designing an algorithm, and then analysing its performance. Algorithmic is a branch of computer science that consists of designing and analyzing computer algorithms. Therefore, this course introduces the basics of the analyses of time and space complexity of some basic algorithms and the notations for describing them. The goal is to provide students with the fundamental knowledge of common algorithms and to deal with a wide variety of computational problems.

Objectives

- i. Describe concepts of algorithm, algorithm design and analysis
- ii. Demonstrate knowledge of algorithm complexity analysis
- iii. Categorize and describe some specific algorithms
- iv. Determine the running time of the basic algorithms in different domains
- v. Analyze the asymptotic performance of algorithms
- vi. Apply important algorithmic design paradigms and methods of analysis.
- vii. Determine worst-case running times of algorithms using asymptotic analysis.
- viii. Determine asymptotic growth rates of algorithms.
- ix. Design and implement efficient divide and conquer algorithms suitable to solve a particular computational problem
- x. Compare algorithms with respect to time and space complexity
- xi. Describe the relative merits of worst, average, and best case analysis of algorithms.
- xii. Analyze recursive and non-recursive algorithms
- xiii. Solve recurrence problems using the master theorem

Compiled: November 2018 Last update: January 2023 Page 2

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Course Outline

1. Fundamentals of Algorithms and Complexity Analysis

- i. Meaning and characteristics of algorithm
- ii. Methods of expressing algorithms
- iii. Basis for studying algorithm

2. Standard Complexity Classes

- o Algorithm Complexity
- o Methods of estimating the runtime of algorithms
- o Asymptotic notations
- o Runtime classification of algorithms
- o Time & Space trade-offs in Algorithm Analysis

3. Non-recursive Algorithms

i. Fundamental Computing Algorithms

- o Linear search
- o Binary Search
- o Bubble sort
- o Selection sort
- o Insertion sort

4. Greedy Algorithms

i. General methods and characteristics of Greedy algorithms

ii. Application of Greedy algorithms

- o Activity selection problem
- o Minimum spanning tree
- o Knapsack problem
- o Unit task scheduling with deadline

5. Divide-and-Conquer Algorithm Design Strategies

i. Quicksort

ii. Binary searchtree

Compiled: November 2018 Last update: January 2023 Page 3

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Textbooks:

1. Introduction to Algorithms – Thomas H. Cormen, Charles E. Leiserson and Ronald L. Rivest
2. Introduction to the Design and Analysis of Algorithms – Anany Levitin

Contacts:

Grading:

(i) Assignments-----12mks (or 0 for copying)

- (ii) Attendance.....3mks (70% total attendance)
- (iii) Test-----15mks (or 0 for cheating)
- (iv) Exam-----70mks
- (v) Total-----100%

Note: Late assignment submission will be graded as mark - x, where x is the number of days of late submission.

Compiled: November 2018 Last update: January 2023 Page 4

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Lecture Schedule

Week Topic

1 Students' registration

2 Fundamentals of Algorithms and Complexity Analysis

- i. Meaning and characteristics of algorithm
- ii. Methods of expressing algorithms
- iii. Basis for studying algorithm

3-5 Standard Complexity Classes

- o Algorithm Complexity
- o Methods of estimating the runtime of algorithms
- o Asymptotic notations
- o Runtime classification of algorithms
- o Time & Space trade-offs in Algorithm Analysis

6-8 Non-recursive Algorithms

- i. Fundamental Computing Algorithms
 - o Linear search
 - o Binary Search
 - o Bubble sort
 - o Selection sort
 - o Insertion sort

9 - 11 Greedy Algorithms

- i. General methods and characteristics of Greedy algorithms
- ii. Application of Greedy algorithms
 - o Activity selection problem
 - o Minimum spanning tree
 - o Knapsack problem
 - o Unit task scheduling with deadline

12 Divide-and-Conquer algorithm Design Strategies

- i. Quicksort
- ii. Binary search tree

13 Test and revision

14 - 15 Examination

Compiled: November 2018 Last update: January 2023 Page 5

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

1. Basic Concepts of Algorithms

1.1 What is Algorithm?

□ An algorithm is a finite sequence of steps expressed for solving a problem.

□ An algorithm can be defined as “a process that performs some sequence of operations in order to solve a given problem”.

□ Algorithm is a step-by-step procedure, which defines a set of instructions to be executed in a certain order to get the desired output.

A procedure can only be called an algorithm to solve a particular problem if it has the following characteristics.

- i. Independence – An algorithm should have step-by-step directions, which should be independent of any programming code.
- ii. Feasibility – It should be feasible within the available resources.
- iii. Input/Output – An algorithm should have 0 or more well-defined inputs and one or more well-defined outputs.
- iv. Finiteness – Algorithm must terminate after a finite number of steps
- v. Unambiguous – Algorithm should be clear and unambiguous. Each step (or phase) and its input/output should be clear and must lead to only one meaning.

Algorithms are used for calculation, data processing, and many other fields.

Algorithms are generally created independent of underlying languages, i.e. an algorithm can be implemented in more than one programming language.

□ Benefits of using Algorithms

The use of algorithms provides a number of benefits.

□ One of these benefits is in the development of the procedure itself, which involves identification of the processes, major decision points, and variables necessary to solve the problem. Identification of the processes

Compiled: November 2018 Last update: January 2023 Page 6

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

and decision points reduces the task into a series of smaller steps of more manageable size.

□ Problems that would be difficult or impossible to solve in entirety can be approached as a series of small, solvable sub-problems.

□ An algorithm serves as a mnemonic device and helps ensure that variables or parts of the problem are not ignored. Presenting the solution process as an algorithm allows more precise communication. Finally, separation of the procedure steps facilitates division of labour and

development of expertise.

□ A final benefit of the use of an algorithm comes from the improvement it makes possible. As results are compared with goals overtime, the existence of a specified solution process allows identification of weaknesses and errors in the process.

o Reduction of a task to a specified set of steps or algorithm is an important part of analysis, control and evaluation.

1.2 Reasons for Using Algorithm

Three reasons for using algorithms are efficiency, abstraction and reusability.

□ Efficiency: Certain types of problems, like sorting, occur often in computing. Efficient algorithms must be used to solve such problems considering the time and cost factor involved in each algorithm.

□ Abstraction: Algorithms provide a level of abstraction in solving problems because many seemingly complicated problems can be distilled into simpler ones for which well known algorithms exist. Once we see a more complicated problem in a simpler light, we can think of the simpler problem as just an abstraction of the more complicated one.

o For example, imagine trying to find the shortest way to route a packet between two gateways in an internet. Once we realize that

Compiled: November 2018 Last update: January 2023 Page 7

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

this problem is just a variation of the more general shortest path problem, we can solve it using the generalised approach.

□ Reusability: Algorithms are often reusable in many different situations. Since many well-known algorithms are the generalizations of more complicated ones, and since many complicated problems can be distilled into simpler ones, an efficient means of solving certain simpler problems potentially lets us solve many complicated problems.

1.3 Methods of Expressing Algorithms

Algorithms can be expressed in many different notations, including

o natural languages,

o pseudocode,

o flowcharts and

o Programming languages

Natural language expressions of algorithms tend to be verbose and ambiguous, and are rarely used for complex or technical algorithms.

Pseudocode and flowcharts are structured ways to express algorithms that avoid many ambiguities common in natural language statements, while remaining independent of a particular implementation language.

Pseudocode is a mixture of a natural language and programming language-like constructs. Pseudocode is usually more precise than natural language, and its usage often yields more succinct algorithm descriptions. The running time of an algorithm can be estimated in a more general manner by using Pseudocode to represent the algorithm as a set of fundamental operations which can then be counted.

Flowcharts are used in analyzing, designing, documenting or managing a process or program in various fields. The purpose of all flow charts is to

Compiled: November 2018 Last update: January 2023 Page 8

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

communicate how a process works or should work without any technical or group specific jargon.

Programming languages are primarily intended for expressing algorithms in a form that can be executed by a computer, but are often used to define or document algorithms.

Sometimes it is helpful in the description of an algorithm to supplement small flowcharts with natural language and/or arithmetic expressions written inside block diagrams to summarize what the flowcharts are accomplishing.

□ Some examples of how to express Algorithms

1. Algorithm to sum the numbers between 1 and 10 inclusive. This is represented in three forms – natural language, pseudo code and flowchart

A: Natural Language

1. Initialize sum
2. Repeat steps 3 and 4 ten times
3. Read number
4. Add number to sum
5. Print sum

Compiled: November 2018 Last update: January 2023 Page 9

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

B. Pseudo code

Input: Integer numbers between 1 and 10 inclusive

Output: Sum of ten integer numbers

1. $Sum \leftarrow 0$
2. for $i \leftarrow 1$ to 10
3. Read number
4. $Sum \leftarrow sum + i$
5. end for
6. Print Sum

C. Flowchart Representation

Start

Read Num

Sum = sum + Num

Yes

More?

No

Print Sum

Stop

Compiled: November 2018 Last update: January 2023 Page 10

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

2. Finding the largest element in array

A. Using Natural language

1. Assign the first element of the array as the largest

2. Repeat step 3 starting from the second element to the last element in the array

3. Swap, if the next element is greater than the element in the previous position

B. Using Pseudocode

Algorithm Max (A, n)

Input: An array A of n integer

Output: Largest element in A

1. Largest $\leftarrow$ A[1]

2. For i $\leftarrow$ 2 to n do

3. if largest < A[i] then largest $\leftarrow$ A[i]

4 End for

Pseudo-code provides a way of expressing algorithms in a way that is independent of any programming language. It abstracts away other program details such as the type system and declaring variables and arrays. Some points to note are:

- The statement blocks are determined by indentation, rather than {and} delimiters as in Java.
- Control statements, such as if...then...else and while have similar interpretations to Java.
- A statement $v \leftarrow e$ implies that expression e should be evaluated and the resulting value assigned to variable v. Or, in the case of $v1 \leftarrow v2 \leftarrow e$, to variables v1 and v2

Compiled: November 2018 Last update: January 2023 Page 11

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

- All variables should be treated as local to their procedures.
- Arrays indexation is denoted by $A[i]$ and arrays are assumed to be indexed from 1 to N (rather than 0 to $N - 1$, the approach followed by Java).

1.4 Why Study Algorithm?

- i. Computers may be fast, but they are not infinitely fast. And memory may be inexpensive, but it is not free. Computing time is therefore a bounded resource, and so is space in memory. You should use these resources wisely, and algorithms that are efficient in terms of time or space will help do so.
- ii. Algorithms are at the core of most technologies used in contemporary computers. Even an application that does not require algorithmic content at the application level relies heavily upon algorithms.
 - a. Does the application rely on fast hardware? The hardware design used algorithms.
 - b. Does the application rely on graphical user interfaces? The design of any GUI relies on algorithms.
 - c. Does the application rely on networking? Routing in networks relies heavily on algorithms.
 - d. Was the application written in a language other than machine code? Then it was processed by a compiler, interpreter, or assembler, all of which make extensive use of algorithms.
- iii. Furthermore, with the ever-increasing capacities of computers, we use them to solve larger problems than ever before. It is at larger problem sizes that the differences in efficiency between algorithms become particularly prominent.

- iv. Having a solid base of algorithmic knowledge and technique is one characteristic that separates the truly skilled programmers from the

Compiled: November 2018 Last update: January 2023 Page 12

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

novices. With modern computing technology, you can accomplish some tasks without knowing much about algorithms, but with a good background in algorithms, you can do much, much more.

Exercises

1. A program to find the n th term of an arithmetic progression is to be written, given the first term (a) and the common difference (d). Write the pseudo code for the program.
2. A supermarket gives a discount of 15% for purchases greater than or

equal to five thousand naira. For any other purchase amount, the discount is 5%. A program is to be written to compute a customer's bill given the amount purchased as input. The program is to calculate and output for at least 2000 customers and a customer gets discount only if payment was by cash. Write a pseudo code for the programming problem.

3. Write an algorithm to compute the sum of the squares of integers from 1 to 50 and also draw the corresponding flowchart. Assuming that you have to code the above given problem, show the steps involved in program design.

Compiled: November 2018 Last update: January 2023 Page 13

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

2. Standard Complexity Classes

Analysis of algorithms is the theoretical study of computer program performance and resource usage, and is often practised abstractly without the use of specific programming language or implementation.

□ The practical goal of algorithm analysis is to predict the performance of different algorithms in order to guide program design decisions.

Inefficient Algorithms can significantly impact system performance.

- o In time-sensitive applications, an algorithm taking too long to run can render its results outdated or useless.

- o An inefficient algorithm can also end up requiring an uneconomical amount of computing power or storage in order to run, again rendering it practically useless.

2.1 Algorithm Complexity

The complexity (Computational complexity) of an algorithm describes the efficiency of the algorithm in terms of the amount of the memory required to process the data and the processing time. Algorithmic complexity is concerned about its performance, how fast or slow it works.

□ Most algorithms do not perform the same in all cases; normally an algorithm's performance varies with the data passed to it.

Complexity of an algorithm is analyzed in two perspectives: time and space.

(i) Time Complexity

It's a function describing the amount of time required to run an algorithm in terms of the size of the input.

Compiled: November 2018 Last update: January 2023 Page 14

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

"Time" can mean the number of memory accesses performed, the number of comparisons between integers, the number of times some inner loop is executed, or some other natural unit related to the amount of real time the

algorithm will take.

For a given problem, we characterize the input size, n , appropriately:

- o Sorting – The number of items to be sorted
- o Graphs – The number of vertices and/or edges
- o Numerical – The number of bits needed to represent a number

The choice of an input size greatly depends on the elementary operation; the most relevant or important operation of an algorithm.

- o Comparisons
- o Additions
- o Multiplications

Indeed, for small values of n , most functions will be very similar in running time. Only for sufficiently large n do differences in running time become apparent.

Types of Complexity

□ Worst-case complexity: The worst-case efficiency of an algorithm is its efficiency for the worst-case input of size n , which is an input (or inputs) of size n for which the algorithm runs the longest among all possible inputs of that size.

Worst-case analysis answers questions such as - what is the longest an algorithm can run for a problem of size n ?

Or, among all inputs of the same size, what is the maximum running time? Computer scientists mostly use worst-case analysis

Compiled: November 2018 Last update: January 2023 Page 15

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The worst-case analysis provides very important information about an algorithm's efficiency by bounding its running time from above. In other words, it guarantees that for any instance of size n , the running time will not exceed $C \text{ worst}(n)$ it's running time on the worst-case inputs.

For example, in a linear search, the worst case is when the target item is at the end of the list or not in the list at all. Then the algorithm must visit every item and perform n iterations for a list of size n . Thus, the worst-case complexity of a linear search is $O(n)$.

There are four reasons why algorithms are generally analyzed by their worst case:

□ Many algorithms perform to their worst case a large part of the time. For example, the worst case in searching occurs when we do not find what we are looking for at all. This frequently happens in database applications.

□ The best case is not very informative because many algorithms perform exactly the same in the best case. For example, nearly all searching algorithms can locate an element in one inspection at best, so analyzing this case does not tell us much.

□ Determining average-case performance is not always easy. Often it is difficult to determine exactly what the “average case” even is; since we can seldom guarantee precisely how an algorithm will be exercised.

□ The worst case gives us an upper bound on performance. Analyzing an algorithm’s worst case guarantees that it will never perform worse than what we determine. Therefore, we know that the other cases must perform at least better than the worst case.

Compiled: November 2018 Last update: January 2023 Page 16

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

□ Average-case complexity: Among all inputs of the same size, what is the average running time? The average is computed assuming some probability distribution that describes the likelihood that a particular input will arise.

Average-case answers questions such as - on average how fast does an algorithm run for a problem of size N ?

To determine the average case, for example in a linear search, add the number of iterations required to find the target at each possible position and divide the sum by n . Thus, the algorithm performs $(n + n - 1 + n - 2 + \dots + 1) / n$, or $(n+1)/2$ iterations.

For very large n , the constant factor of $/2$ is insignificant, so the average complexity is still $O(n)$ as in worst case.

The best-case efficiency of an algorithm is its efficiency for the best-case input of size n , which is an input (or inputs) of size n for which the algorithm runs the fastest among all possible inputs of that size.

Best-case analysis answers the question such as - what is the fastest an algorithm can run for a problem of size N ?

For example, in a linear search, the best case is when the algorithm finds the target at the first position, after making one iteration, for an $O(1)$ complexity.

□ Why know Time efficiency of algorithms?

We like to know the time efficiency of a program for several reasons:

- o To get an estimate of how long a program will run. (Waiting for it to finish may not be feasible!)

Compiled: November 2018 Last update: January 2023 Page 17

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

- o To get an estimate of how large an input the program can handle without taking too long.
- o To compare the efficiency of different programs for solving the same problem.

(ii) Space Complexity

It's a function describing the amount of memory an algorithm takes in terms of the size of input to the algorithm. We often speak of "extra" memory needed, not counting the memory needed to store the input itself. Again, we use natural (but fixed-length) units to measure this.

Space complexity is sometimes ignored because the space used is minimal and/or obvious, however sometimes it becomes as important an issue as time.

Space is measured by counting the maximum memory space required by the algorithm. The space required by an algorithm is equal to the sum of the following two components:

- A fixed part that is a space required to store certain data and variables that are independent of the size of the problem. For example, simple variables and constants used, program size, etc.
- A variable part is a space required by variables, whose size depends on the size of the problem. For example, dynamic memory allocation, recursion stacks space, etc.

Compiled: November 2018 Last update: January 2023 Page 18

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

2.2 Methods of estimating the Runtime of algorithm

a. Use of computer Clock

One way to measure the time cost of an algorithm is to use the computer's clock to obtain an actual run time. This process is called benchmarking or profiling

- o It starts by determining the time for several different data sets of the same size and then calculates the average time.
- o Next, similar data are gathered for larger and larger data sets. After several such tests, enough data are available to predict how the algorithm will behave for a data set of any size.

The program uses the `time()` function to track the running time.

This function returns the number of seconds that have elapsed between the current time on the computer's clock and start time.

Thus, the difference between the results of two calls of `time.time()`

represents the elapsed time in seconds.

e.g.

```
Start = time.timer()
```

```
//start of the algorithm
```

```
The algorithm
```

```
//End of the algorithm
```

```
Elapsed = time.timer - start
```

This method permits accurate predictions of the running times of many algorithms.

However, there are some problems with this technique:

- Different hardware platforms have different processing speeds, so the running times of an algorithm differ from machine to machine.

Compiled: November 2018 Last update: January 2023 Page 19

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

- Also, the running time of a program varies with the type of operating system that lies between it and the hardware.

- Finally, different programming languages and compilers produce code whose performance varies. For example, an algorithm coded in C usually runs slightly faster than the same algorithm in Python byte code.

- The “coding efficiency” of the programmer who converts the algorithm to a program can have a tremendous impact as well.

- It is impractical to determine the running time for some algorithms with very large data sets. For some algorithms, it doesn't matter how fast the compiled code or the hardware processor is. They are impractical to run with very large data sets on any computer.

Thus, predictions of performance generated from the results of timing on one hardware or software platform generally cannot be used to predict potential performance on other platforms.

So it is not an independent measure of the algorithm, but rather a measure of the implementation, the machine and the instance.

b. Counting Instructions

Another technique used to estimate the efficiency of an algorithm is to count the instructions executed with different problem sizes. These counts provide a good predictor of the amount of abstract work performed by an algorithm, no matter what platform the algorithm runs on.

Keep in mind, however, that when you count instructions, you are counting the instructions in the high-level code in which the algorithm is written, not instructions in the executable machine language program.

Compiled: November 2018 Last update: January 2023 Page 20

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

When analyzing an algorithm in this way, you distinguish between two classes of instructions:

- i. Instructions that execute the same number of times regardless of the problem size
 - ii. Instructions whose execution count varies with the problem size
- e.g. instructions within loops

Note: The first class of instructions does not affect significantly in algorithm analysis. The instructions in the second class normally are found in loops or recursive functions. In the case of loops, focus is also on instructions performed in any nested loops or, more simply, just the number of iterations that a nested loop performs.

Of primary consideration when estimating an algorithm's performance is the number of basic operations required by the algorithm to process an input of a certain size.

□ Size is often the number of inputs processed. For example, when comparing sorting algorithms, the size of the problem is typically measured by the number of records to be sorted.

□ A basic operation must have the property that its time to complete does not depend on the particular values of its operands.

o Adding or comparing two integer variables are examples of basic operations in most programming languages.

o Summing the contents of an array containing n integers is not, because the cost depends on the value of n (i.e., the size of the input).

Primitive (basic) operations include:

- Calling a method and returning from a method
- Performing an arithmetic operation (e.g. addition)

Compiled: November 2018 Last update: January 2023 Page 21

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

- Comparing two numbers
- Assigning a value to a variable
- Indexing an array
- Evaluating an expression, etc

Because the most important factor affecting running time is normally size of the input, for a given input size n we often express the time T to run the algorithm as a function of n , written as $T(n)$.

Time is measured by counting the number of key operations such as

comparisons in a sorting algorithm.

c. Asymptotic complexity measure

Time of algorithm can also be measured without necessarily counting the exact number of steps of a program, but how that number grows with the size of the input to the program. This approach is based on the asymptotic complexity measure.

The limiting behaviour of the complexity as size increases is called the asymptotic time complexity.

It is the asymptotic complexity of an algorithm, which ultimately determines the size of problems that can be solved by the algorithm. The asymptotic complexity is written using the appropriate asymptotic notations discussed in the next section.

Compiled: November 2018 Last update: January 2023 Page 22

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

2.3 Asymptotic Notations

Asymptotic analysis refers to the study of an algorithm as the input size “gets big” or reaches a limit. Asymptotic notation gives us a method for classifying functions according to their rate of growth.

It allows the analysis of an algorithm’s running time by identifying its behaviour as the input size for the algorithm increases. This is also known as an algorithm’s growth rate.

Asymptotic analysis attempts to:

- o estimate the resource consumption of an algorithm
- o compare the relative costs of two or more algorithms for solving the same problem.

Asymptotic analysis also gives algorithm designers a tool for estimating whether a proposed solution is likely to meet the resource constraints for a problem before they implement an actual program.

Let us imagine an algorithm as a function f , n as the input size, and $f(n)$ being the running time. So for a given algorithm f , with input size n you get some resultant run time $f(n)$. This can be represented in a graph where the y axis is the runtime, x axis is the input size, and plot points are the resultants of the amount of time for a given input size.

Compiled: November 2018 Last update: January 2023 Page 23

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Primarily we are interested only in the growth rate of f , which describes how quickly the algorithm’s performance will degrade as the size of data it processes becomes arbitrarily large.

(i) Upper bound: It indicates the upper or highest growth rate that the algorithm can have. A special notation, called big-Oh notation is adopted. It is usually written in relation to the input size such as $O(n)$. The $O(n)$ is read as "Oh of n " or "big Oh of n ". The $O(n)$ measures the worst case time complexity or the longest amount of time an algorithm can possibly take to complete.

The following is a precise definition for an upper bound:

For $f(n)$ a non-negatively valued function, $f(n)$ is in set $O(g(n))$ if there exist two positive constants c and n_0 such that $f(n) \leq cg(n)$ for all $n > n_0$.

In other words,

$f(n)$ is said to be $O(g(n))$

if there exists a natural number n_0 and a constant $c > 0$ such that

$\forall n \geq n_0; f(n) \leq cg(n)$:

The definition says that for all inputs of the type in question (such as the worst case for all inputs of size n) that are large enough (i.e., $n > n_0$), the algorithm always executes in less than $cg(n)$ steps for some constant c .

NOTE: i. $f(n)$ represents the true running time of the algorithm.

ii. $g(n)$ is some expression for the upper bound (i.e. $g(n)$ is an arbitrary time complexity you are trying to relate to your algorithm).

iii. Constant n_0 is the smallest value of n for which the claim of an upper bound holds true

Compiled: November 2018 Last update: January 2023 Page 24

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

iv. c encapsulates many factors eg. Implementation details of algorithm, machine, technology used etc

This can be represented graphically as shown below

It should be emphasized, however, that this does not mean that the running time is always as large as $cg(n)$, even for large input sizes. Thus, the O -notation provides an upper bound on the running time

Rules for using big- O :

The most important property is that big- O gives an upper bound only. If an algorithm is $O(n^2)$, it doesn't have to take n^2 steps (or a constant multiple of n^2). But it can't take more than n^2 . So any algorithm that is $O(n)$, is also an $O(n^2)$ algorithm. Think of big- O as being like " $<$ ". Any number that is $< n$ is also $< n^2$.

When faced with a complicated function like $3n^2 + 4n + 5$, we just replace it with $O(f(n))$, where $f(n)$ is as simple as possible. In this particular example we'd use $O(n^2)$, because the quadratic portion of the sum dominates the rest of the function.

Here are some rules that help simplify functions by omitting dominated terms:

Compiled: November 2018 Last update: January 2023 Page 25

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

□ We can ignore constant terms because as the value of n becomes larger and larger, eventually constant terms will become insignificant. For example, if $T(n) = n + 50$ describes the running time of an algorithm, and n , the size of data it processes, is only 1024, the constant term in this expression constitutes less than 5% of the running time.

□ We can ignore multiplicative constants because they too will become insignificant as the value of n increases. For example, $14n^2$ becomes n^2 .

□ We need to consider the highest-order term because as n increases higher-order terms quickly outweigh the lower-order terms. That is, na dominates nb if $a > b$. For instance, n^2 dominates n .

□ We also must note that an exponential term dominates any polynomial term in the expression. For example, 3^n dominates n^5 (it even dominates $2n$).

□ Similarly, a polynomial term dominates any logarithmic term used in the expression. For example, n dominates $(\log n)^3$. This also means, for example, that n^2 dominates $n \log n$.

Suppose we have an algorithm whose running time is described by the function $T(n) = 3n^2 + 10n + 10$.

Using the rules of O -notation, this function can be simplified to:

$$O(T(n)) = O(3n^2 + 10n + 10) = O(3n^2) = O(n^2)$$

We can verify this quantitatively by computing the percentage of the overall running time that each term accounts for as n increases. For example, when $n = 10$, we have the following:

$$\text{Running time for } 3n^2: 3(10)^2 / (3(10)^2 + 10(10) + 10) = 73.2\%$$

$$\text{Running time for } 10n: 10(10) / (3(10)^2 + 10(10) + 10) = 24.4\%$$

$$\text{Running time for } 10: 10 / (3(10)^2 + 10(10) + 10) = 2.4\%$$

Compiled: November 2018 Last update: January 2023 Page 26

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

This indicates that the term n^2 will be the one that accounts for the most of the running time as n grows arbitrarily large.

(ii) Lower Bounds: describe the least amount of a resource that an algorithm needs for some class of input (i.e. the worst-, average-, or best-case input of size n). The lower bound for an algorithm is denoted by the symbol Ω , pronounced “big-Omega” or just “Omega.” For example, $\Omega(n)$ is read as “Omega of n ” or “big-Omega of n ”. The notation $\Omega(n)$ measures the best case time complexity or the best amount of time an algorithm can possibly take to complete.

For $f(n)$ a non-negatively valued function, $f(n)$ is in set $(g(n))$ if there exist two positive constants c and n_0 such that $f(n) \geq cg(n)$ for all $n > n_0$.

$\Omega(g(n)) = \{f(n): \text{there exist positive constants } c \text{ and } n_0 \text{ such that}$

$0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\}$

$\Omega(g(n))$ is the set of all functions with a larger or same order of growth as $g(n)$.

Compiled: November 2018 Last update: January 2023 Page 27

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

(iii) The big-Theta notation - Θ : When the upper and lower bounds are the same within a constant factor, we indicate this by using Θ (big-Theta) notation. $\Theta(g(n))$ is the set of all functions with the same order of growth as $g(n)$.

In general, we say that the running time of an algorithm is of order $\Theta(g(n))$ if whenever the input size is equal to or exceeds some threshold n_0 , its running time can be bounded below by $c_1g(n)$ and above by $c_2g(n)$, where $0 < c_1 \leq c_2$.

Thus, this notation is used to express the exact order of an algorithm, which implies an exact bound on its running time.

$\Theta(g(n)) = \{f(n): \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that}$

$0 \leq c_1g(n) \leq f(n) \leq c_2g(n) \text{ for all } n \geq n_0\}$

These two constants - c_1 and c_2 - encapsulate many factors pertaining to the details of the implementation of the algorithm and the machine and technology used.

Represented graphically as:

It may be helpful to think of O as similar to $\leq$, Ω as similar to $\geq$ and Θ as similar to $=$.

2.4 Runtime Classification of Algorithms

The running time of an algorithm on a particular input is the number of primitive operations or “steps” executed.

Compiled: November 2018 Last update: January 2023 Page 28

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Constant time: An algorithm is said to be constant time (written as $O(1)$ time) if the value of $T(n)$ is bounded by a value that does not depend on the size of the input. This means that the algorithm requires the same fixed number of steps regardless of the size of the task.

For example, accessing any single element in an array takes constant time as only one operation has to be performed to locate it

Examples: (i):

(ii):

Note: Each of the assignment statements takes constant time. The for loop in the second example involves fixed number of operations and hence requires constant time.

More general examples include:

- array: accessing any element
- fixed-size stack: push and pop methods

Logarithmic ($\log n$): Implies that the running time of the algorithm is proportional to the logarithm of the input size. The algorithm of this type does not use the whole input.

Compiled: November 2018 Last update: January 2023 Page 29

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

o This occurs in programs that solve a big problem by transforming it into a smaller problem, cutting the size by some constant fraction.

o Whenever n doubles, $\log n$ increases by a constant, but $\log n$ does not double until n increases to n^2 . It always increases, but at a slower rate as n increases.

A logarithmic function is the inverse of an exponential function, i.e. $b^x = n$ is equivalent to

$$x = \log_b n$$

Logarithm denotes a quantity representing the power to which a fixed number (the base) must be raised to produce a given number. Thus

$$n = 2^k$$

is represented in logarithmic form as $\log_2 n = k$

What is important here is that the running time grows in proportion to the logarithm of the input (in this case, log to the base 2)

Consider the following example

```
Sum = 0;
for (k = 1; k <= n; k*=2) // do log n times.
sum ++;
```

If n is 8, the loop runs $\log(8)$ times i.e. 3 times.

More general examples of logarithm algorithm include

- o Binary search
- o Searching a tree data structure

Linear time (n): A linear-time algorithm is one that takes a number of steps directly proportional to the size of the input. In other words, if the size of the input doubles, the number of steps doubles. This means that the algorithm requires a number of steps proportional to the size of the task. It is represented as $O(n)$ and referred to as Linear time.

Compiled: November 2018 Last update: January 2023 Page 30

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Basic structure is:

```
for (i = 0; i < N; i++) {
```

Sequence of statements of $O(1)$

The loop executes N times, so the total time is $N \cdot O(1)$ which is $O(N)$.

Examples:

(i)

```
for (int i = 0; i < data.Length; i++)
```

```
if (data[i] == find)
```

```
return i;
```

(ii):

(iii):

More general examples include

- o Traversing an array.

- o Sequential/Linear search in an array

Compiled: November 2018 Last update: January 2023 Page 31

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Quasi-linear or log-linear ($n \log n$): The running time grows in proportion to $n \log n$ of the input. It is a combination of $O(n)$ and $O(\log n)$ running time.

Example 1:

```
for (int i = 1; i <= n; i++){
```

```
for(int j = 1; j < 5; j = j * 2) {
```

```
System.out.println("i" + i + " and j" + j);
```

For example, if n is 8, then this algorithm will run $8 * \log(8) = 8 * 3 = 24$ times.

Example 2:

```
Sum = 0;
```

```
for (k = 1; k <= n; k*=2) // do log n times.
```

```
for (j = 1; j <= n; j++) //do n times
```

```
sum ++;
```

The inner loop executes n times while the outer loop executes $\log n + 1$ times because on each iteration k is multiplied by two until it reaches n . The total runtime is $n \log n$.

Quadratic time (n^2):. That is, the number of operations is proportional to the size of the task squared. It means the running time of the algorithm on an input of size n is limited by the quadratic function of n . It is represented as $O(n^2)$ and referred to as quadratic time

- o Quadratic running times typically arise in algorithms that process all pairs of data items (perhaps in a double nested loop) whenever n doubles, the running time increases four fold.
- o When the running time of an algorithm is quadratic, it is practical for use only on relatively small problems

Compiled: November 2018 Last update: January 2023 Page 32

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Examples:

(i):

```
for (i = 0; i < N; i++) {
  for (j = 0; j < N; j++) {
    .....
```

There are 2 nested for loops, each of which runs n times. The innermost loop therefore executes $n \cdot n = n^2$ times

(ii):

```
for (int i = 0; i < n; i++) // Loop1
for (int j = i + 1; j > i; j--) // Loop2
for (int k = n; k > j; k--) // Loop3
system.out.println("*");
```

Loop2 runs a constant number of times (exactly once) every loop1 iteration and does not affect the time complexity (although there are three for loops, only two affect the running time, since they depend on the input size). We can carefully trace the values of j in each loop1/loop2 iteration to find the number of times the innermost loop iterates.

(iii):

Consider the following code:

What is the running time for this code fragment? Clearly it takes longer to run when n is larger.

Compiled: November 2018 Last update: January 2023 Page 33

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

- o The basic operation in this example is the increment operation for variable sum . The total number of increment operations is n^2 . Thus, we say that the running time is $T(n) = n^2$

Cubic (n^3): Similarly, an algorithm that process triples of data items (perhaps in a triple-nested loop) has a cubic running time. Whenever n doubles, the running time increases eight fold.

- o It is practical for use only on small problems.

Example: The code to compute the matrix product $C = A * B$ is given below:

```

for (i = 1; i<=n; i++)
for (j = 1; j<=n; j++)
C[i, j] = 0;
for (k = 1; k<=n; k++)
C[i, j] = C[i, j] + A[i, k] * B[k, j];

```

There are 3 nested for loops, each of which runs n times. The innermost loop therefore executes $n \times n \times n = n^3$ times. The innermost statement, which contains a scalar sum and product takes constant $O(1)$ time. So the algorithm overall takes $O(n^3)$ time.

Exponential (2^n): Few algorithms with exponential running time are likely to be appropriate for practical use, such algorithms arise naturally as “brute-force” solutions to problems. Whenever n doubles, the running time squares.

Compiled: November 2018 Last update: January 2023 Page 34

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Polynomial (n^k): More generally, if an algorithm is $O(n^k)$ for constant k it is called a polynomial-time algorithm. Thus, quadratic and cubic running time algorithms are polynomial.

NOTE: There is difference between an exponential function and a polynomial. The function $p(x) = x^3$ is a polynomial. Here the “variable”, x , is raised to some constant power (i.e. 3 in the example). The function $f(x) = 3^x$ is an exponential function where the variable x is the exponent.

The graph below shows the growth of the algorithm running time with given input size. The growth rate for an algorithm is the rate at which the cost of the algorithm grows as the size of its input grows

Exercise

Using the given inputs in the table below, compute the growth rate of the three algorithms with $n \log n$, n^2 and $\log n$ runtime respectively and plot the graph to show the growth rate of the algorithm

Compiled: November 2018 Last update: January 2023 Page 35

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Input $\log n$

Size $n \log n$ n^2

256

1024

2048

4096

8192

Compiled: November 2018 Last update: January 2023 Page 36

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

3. Non-recursive Algorithms

A non-recursive algorithm or simply an iterative function is one that loops to repeat some part of the code; contrast to a recursive function that calls itself again to repeat the code.

o Using a simple for loop to display the numbers from one to ten is an iterative process.

In a recursive algorithm, current output depends on previous output(s) and input(s) but this is not the case in non-recursive algorithms.

Examples:

(i) Recursive Algorithm

Algorithm F(n)

if $n = 0$ then return 1 // base case

else $F(n-1) \cdot n$ // recursive call

(ii) Non-recursive Algorithm

Algorithm Binary(n)

count:= 1;

while $n > 1$ do

count:= count + 1;

$n := \lfloor n/2 \rfloor$;

end

return count;

Compiled: November 2018 Last update: January 2023 Page 37

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

3.1 Analysis of Non-recursive Algorithms

After developing pseudo-code for an algorithm, we wish to analyze its efficiency as a function of the size of the input, n in terms of how many times the elementary operation is performed. Here is a general strategy:

- Decide on parameter n indicating input size
- Identify algorithm's basic operation.
- Check whether the number of time the basic operation is executed depends on some additional property of the input. If so, determine worst, average, and best case for input of size n
- Count the number of operations. Set up a summation corresponding to the number of elementary operations
- Simplify the equation to get as simple of a function $f(n)$ as possible.
- Establish the efficiency class to which the algorithm belongs

NOTE: The core of the algorithm analysis is to find out how the number of the basic operations depends on the size of the input.

Examples of primitive operations:

- o Evaluating an expression e.g. $x^2 + ey$
- o Assigning a value to a variable e.g. $cnt \leftarrow cnt + 1$
- o Indexing into an array e.g. $A[5]$
- o Calling a method e.g. $mySort(A,n)$
- o Returning from a method e.g. $return(cnt)$

In the following search algorithm, there are six basic operations which are marked with a box and labelled 1 – 6.

Compiled: November 2018 Last update: January 2023 Page 38

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Let the $x.length = n$.

Assume a worst case scenario for the above search algorithm:

Number of times operation 1 executes = 1

Number of times operation 2 executes = $n + 1$

Number of times operation 3 executes = n

Number of times operation 4 executes = n

Number of times operation 5 and 6 execute = 1

Total number of operations = $3n + 3$

There are four general rules for counting the operations:

Rule 1: for loops - the size of the loop $\times$ the running time of the body

The running time of a for loop is at most the running time of the statements inside the loop $\times$ the number of iterations.

For ($i = 0$; $i < n$; $i++$)

sum = sum + i;

Example:

a). Find the running time of statements when executed only once:

Compiled: November 2018 Last update: January 2023 Page 39

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The statements in the loop heading have fixed number of operations, hence they have constant running time $O(1)$ when executed only once.

The statement in the loop body has fixed number of operations; hence it has a constant running time when executed only once.

b). Find how many times each statement is executed.

For ($i = 0$; $i < n$; $i++$) // $i = 0$; executed only once: $O(1)$

// $i < n$; $n + 1$ times $O(n)$

// $i++$ n times $O(n)$

// total time of the loop header: $O(1) + O(n) +$

$O(n) = O(n)$

sum = sum + i; // executed n times, $O(n)$

The loop header plus the loop body will give: $O(n) + O(n) = O(n)$.

Loop running time is: $O(n)$

Rule 2: Nested loops - the product of the size of the loops X the running time of the body

The total running time is the running time of the inside statements times the product of the sizes of all the loops

```
sum = 0; for( i = 0; i < n; i++)
```

```
for( j = 0; j < n; j++)
```

```
sum++;
```

Applying Rule 1 for the nested loop (the 'j' loop) we get $O(n)$ for the body of the outer loop.

The outer loop runs n times, therefore the total time for the nested loops will be

$O(n) * O(n) = O(n*n) = O(n^2)$

What happens if the inner loop does not start from 0?

```
sum = 0; for( i = 0; i < n; i++)
```

Compiled: November 2018 Last update: January 2023 Page 40

CSC315 - Algorithms & Complexity Analysis - Orhionkpaiyo, B. C

```
for( j = i; j < n; j++)
```

```
sum++;
```

Here, the number of the times the inner loop is executed depends on the value of i

i = 0, inner loop runs n times

i = 1, inner loop runs (n-1) times

i = 2, inner loop runs (n-2) times

i = n - 2, inner loop runs 2 times

i = n - 1, inner loop runs once.

Thus we have: $(1 + 2 + \dots + n) = n*(n+1)/2 = O(n^2)$

General rule for nested loops:

Running time is the product of the size of the loops times the running time of the body.

Example:

```
sum = 0;
```

```
for( i = 0; i < n; i++)
```

```
for( j = 0; j < 2n; j++)
```

```
sum++;
```

We have one operation inside the loops, and the product of the sizes is $2n^2$

Hence the running time is $O(2n^2) = O(n^2)$

Note: if the body contains a function call, its running time has to be taken into consideration.

Compiled: November 2018 Last update: January 2023 Page 41

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

```
sum = 0;
```

```
for( i = 0; i < n; i++)
```

```
for( j = 0; j < n; j++)
```

```
sum = sum + function(sum);
```

Assume that the running time of `function(sum)` is known to be $\log(n)$.

Then the total running time will be $O(n^2 * \log(n))$

The code to compute the matrix product $C = A * B$ is given below.

```
for (i = 1; i<=n; i++)
```

```
for (j = 1; j<=n; j++)
```

```
C[i, j] = 0;
```

```
for (k = 1; k<=n; k++)
```

```
C[i, j] = C[i, j] + A[i, k] * B[k, j];
```

There are 3 nested for loops, each of which runs n times. The innermost loop therefore executes $n*n*n = n^3$ times. The innermost statement, which contains a scalar sum and product takes constant $O(1)$ time. So the algorithm overall takes $O(n^3)$ time.

Rule 3: Consecutive program fragments

The total running time is the maximum of the running time of the individual fragments

```
sum = 0;
```

```
for( i = 0; i < n; i++) // runs O(n) time
```

```
sum = sum + i;
```

Compiled: November 2018 Last update: January 2023 Page 42

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

```
sum = 0;
```

```
for( i = 0; i < n; i++)
```

```
for( j = 0; j < 2*n; j++)
```

```
sum++;
```

The first loop runs in $O(n)$ time, the second - $O(n^2)$ time, the maximum is $O(n^2)$

Rule 4: If statement

```
if C
```

```
S1;
else
S2;
```

The running time is the maximum of the running times of S1 and S2.

3.1.1 General rules for the analysis of programs

In general the running time of a statement or group of statements may be parameterized by the input size and/or by one or more variables. The only permissible parameter for the running time of the whole program is 'n' the input size.

1. The running time of each assignment, read, and write statement can usually be taken to be $O(1)$.
2. The running time of a sequence of statements is determined by the sum rule - i.e. the running time of the sequence is, within a constant factor, the largest running time of any statement in the sequence.

□ Consecutive Statements: S1; S2; S3; ... ; SN, The runtime R of a sequence of statements is the runtime of the statement with the max runtime:

$\max(R(S1), R(S2), \dots, R(SN))$

Compiled: November 2018 Last update: January 2023 Page 43

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

3. The running time of an if-statement is the cost of conditionally executed statements, plus the time for evaluating the condition. The time to evaluate the condition is normally $O(1)$.

The time for an if-then-else construct is the time to evaluate the condition plus the larger of the time needed for the statements executed when the condition is true and the time for the statements executed when the condition is false.

□ Conditional Execution: if (S1) S2; else S3;

The runtime of a conditional execution is

$\max(R(S1), R(S2), R(S3))$

4. The time to execute a loop is the sum, over all times around the loop, the time to execute the body and the time to evaluate the condition for termination (usually the latter is $O(1)$).

Often this time is the product of the number of times around the loop and the largest possible time for one execution of the body, but we must consider each loop separately to be sure.

□ Count inside out: the runtime is the max of the runtime of each statement multiplied by the total number of times each statement is executed

```
for (i = 0; i < N; i++) {
```

```
for (k = i; k < N; k++) {
  j = p[j]*3;
```

Compiled: November 2018 Last update: January 2023 Page 44

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The table below shows the summary of the rules

Type of code Technique

No loops or $O(1)$

recursion

Loop multiply

Sequence Add

Selection Take the maximum

Method call Apply method's time to size of arguments

We conclude this section with this example:

Algorithm innerProduct

Input: Non-negative integer n , two integer arrays

A and B of size n .

Output: The inner product of the two arrays

1. $prod \leftarrow 0$

2 for $i \leftarrow 0$ to $n-1$ do

3 $prod \leftarrow prod + A[i]*B[i]$

4 return $prod$

□ Line 1 is one op (assigning a value).

□ Loop initialization is one op (assigning a value).

□ Line 3 is five ops per iteration (multiplicatio, add, 2 array references, assign).

□ Line 3 is executed n times; total is $5n$.

□ Loop increment is two ops (an addition and an assignment)

□ Loop increment is done n times; total is $2n$.

□ Loop termination test is one op (a comparison $i < n$) each time.

□ Loop termination is done $n+1$ times (n successes, one failure); total is $n+1$.

□ Return is one op.

The total is thus: $1 + 1 + 5n + 2n + (n+1) + 1 = 8n + 4$.

Compiled: November 2018 Last update: January 2023 Page 45

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

3.2 Fundamental Computing Algorithms

3.2.1 Search Algorithms

Searching is the algorithmic process of finding a particular item in a collection

of items.

- o A search typically answers either True or False as to whether the item is present.
- o On occasion it may be modified to return where the item is found.

i). Linear/Sequential Search Algorithm

The basic idea:

□ Starting at the first element, the algorithm sequentially steps through an array examining each element until it locates the value it is searching for or until the array is exhausted.

The Algorithm

Algorithm linearSearch

Input: An array $A[1..n]$ of n elements and an element x .

Output: j if $x = A[j]$, $1 \leq j \leq n$, and 0 otherwise.

1. $j \leftarrow 1$
2. while $(j < n)$ and $(x \neq A[j])$
3. $j \leftarrow j + 1$
4. end while
5. if $x = A[j]$ then return j else return 0

The algorithm scans the entries in A and compares each entry with x .

o If after j comparisons, $1 \leq j \leq n$, the search is successful, i.e., $x = A[j]$, j is returned;

o Otherwise a value of 0 is returned indicating an unsuccessful search.

Compiled: November 2018 Last update: January 2023 Page 46

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The sequential search of a list of integers is illustrated below.

If the item is not in the list, the only way to know it is to compare it against every item present.

o If there are items, then the sequential search requires comparisons to discover that the item is not there.

o In the case where the item is in the list, the analysis is not so straightforward.

There are actually three different scenarios that can occur.

o In the best case we will find the item in the first place we look, at the beginning of the list. We will need only one comparison (i.e. $O(1)$ running time for the best case).

o In the worst case, we will not discover the item until the very last comparison, the n th comparison.

o Worst still, the item is not in the list

The worst case Analysis of the algorithm

The worst case is when there are no matching elements or the first matching element is the last one on the list. In this case, the algorithm makes the largest number of key comparisons among all possible inputs of size n .

Thus:

$$C(n) = n.$$

So its worst-case running time is $O(n)$.

Compiled: November 2018 Last update: January 2023 Page 47

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

ii. Binary Search Algorithm

Binary search is a remarkably efficient algorithm for searching in a sorted array. It is a search that is based on divide-and-conquer strategy.

o Although binary search is clearly based on a recursive idea, it can be easily implemented as a non-recursive algorithm, too.

o It works by comparing a search key K with the array's middle element $A[m]$. If they match, the algorithm stops; otherwise, the same operation is repeated recursively for the first half of the array if $K < A[m]$, and for the second half if $K > A[m]$:

As an example, let us apply binary search to searching for $K = 70$ in the array

The iterations of the algorithm are given in the following table:

The following is pseudo code of the non-recursive version.

Compiled: November 2018 Last update: January 2023 Page 48

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

ALGORITHM BinarySearch($A[0..n - 1]$, K)

Input: An array $A[0..n - 1]$ sorted in ascending order and a search key K

Output: An index of the array's element that is equal to K or -1 if there is no such element

$l \leftarrow 0$; $r \leftarrow n - 1$

while $l \leq r$ do

$m \leftarrow \lfloor (l + r)/2 \rfloor$

if $K = A[m]$

return m

else if

$K < A[m]$

$r \leftarrow m - 1$

else

$l \leftarrow m + 1$

return -1

Alternatively, this can be represented as follows:

```

Low = 0; High = N - 1
while (Low ≤ High)
Mid = (Low + High)/2 //Find the middle index
if (X > A[Mid]) //search second half of array
Low = Mid + 1
else if (X < A[Mid]) // search first half
High = Mid - 1
else return Mid //found X

```

Compiled: November 2018 Last update: January 2023 Page 49

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Illustration:

Compare X with middle item A[mid], go to left half if $X < A[mid]$ and right half if $X > A[mid]$. Repeat.

Complexity Analysis of Binary Search

The standard way to analyze the efficiency of binary search is to count the number of times the search key is compared with an element of the array.

o The worst-case inputs include all arrays that do not contain a given search key, as well as some successful searches.

Recall that each comparison eliminates about half of the remaining items from consideration.

o If we start with n items, about $n/2$ items will be left after the first comparison. After the second comparison, there will be about $n/4$ then, $n/8$, $n/16$, and so on.

o When we split the list enough times, we end up with a list that has just one item. Either that is the item we are looking for or it is not. Either way, we are done.

For an array of size N, it eliminates $\frac{1}{2}$ until 1 element remains.

$N, N/2, N/4, N/8, \dots, 4, 2, 1$

Think of it from the other direction:

– How many times do I have to multiply by 2 to reach N? 1, 2, 4, 8, ..., $N/4, N/2, N$

– Call this number of multiplications "x".

Compiled: November 2018 Last update: January 2023 Page 50

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

$2^x = N$

$X = \log_2 N$

- Binary search is in the logarithmic complexity class.

Detailed Analysis

How many iterations are executed before low > high?

After first iteration: $n/2$ items remaining

2nd iteration: $(n/2)/2 = n/4$

Kth iteration: $n/2^k$ remaining

Worst Case: Last iteration occurs when $n/2^k \geq 1$ and $n/2^{k+1} < 1$ item remaining

i.e. $2^k \leq n$ and $2^{k+1} > n$ [take log of both sides]

Number of iterations is $K \leq \log n$ and $k > \log n - 1$

Worst case running time = $O(\log n)$

3.2.2. Sorting Algorithms

Sorting is ordering a list of objects. We can distinguish two types of sorting.

□ If the number of objects is small enough to fit into the main memory, sorting is called internal sorting.

□ If the number of objects is so large that some of them reside on external storage during the sort, it is called external sorting.

In this section, we will consider the following internal sorting algorithms

□ Bubble sort

□ Insertion sort

□ Selection sort

Compiled: November 2018 Last update: January 2023 Page 51

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

i. Bubble Sort Algorithm

Bubble Sort consists of a simple double for loop.

□ The first iteration of the inner for loop moves through the record array from bottom to top, comparing adjacent keys. If the lower-indexed key's value is greater than its higher-indexed neighbour, then the two values are swapped. Once the smallest value is encountered, this process will cause it to "bubble" up to the top of the array.

□ The second pass through the array repeats this process. However, because we know that the smallest value reached the top of the array on the first pass, there is no need to compare the top two elements on the second pass. Likewise, each succeeding pass through the array compares adjacent elements, looking at one less value than the preceding pass.

The action of the algorithm on the list 42, 20, 17, 13, 28, 14, 23, 15 is illustrated below.

The above is an illustration of Bubble Sort. Each column shows the array after

the iteration with the indicated value of i in the outer for loop. Values above the line in each column have been sorted. Arrows indicate the swaps that take place during a given iteration.

A bubble sort can also be understood if the array to be sorted is envisaged as a vertical column whose smallest elements are at the top and whose largest elements are at the bottom.

Compiled: November 2018 Last update: January 2023 Page 52

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The array is scanned from the bottom up, and two adjacent elements are interchanged if they are found to be out of order with respect to each other.

□ First, items $\text{data}[n-1]$ and $\text{data}[n-2]$ are compared and swapped if they are out of order. Next, $\text{data}[n-2]$ and $\text{data}[n-3]$ are compared, and their order is changed if necessary, and so on up to $\text{data}[1]$ and $\text{data}[0]$. In this way, the smallest element is bubbled up to the top of the array.

□ However, this is only the first pass through the array. The array is scanned again comparing consecutive items and interchanging them when needed, but this time, the last comparison is done for $\text{data}[2]$ and $\text{data}[1]$ because the smallest element is already in its proper position, namely, position 0.

□ The second pass bubbles the second smallest element of the array up to the second position, position 1. The procedure continues until the last pass when only one comparison, $\text{data}[n-1]$ with $\text{data}[n-2]$, and possibly one interchange are performed.

The array [5 2 3 8 1] sorted by bubble sort is shown below.

The sorting activities of the Bubble sort algorithm are summarized as follows:

- Performs multiple passes over the array
- In every pass:

Compiled: November 2018 Last update: January 2023 Page 53

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

- o Compare adjacent elements in the list
- o Exchange the elements if they are out of order
- o Each pass moves the largest (or smallest) elements to the end of the array

□ Repeating this process in several passes eventually sorts the array into ascending (or descending) order

The exchange operation, sometimes called a “swap”. Typically, swapping two elements in a list requires a temporary storage location (an additional memory location). This is illustrated in the diagram below.

The pseudo code of the algorithm is as follows:

```

1: Version 1
bubblesort(data[])
for i = 0 to data.length-2
for j = data.length-1 downto i+1
if elements are out of order swap elements in
position j and j-1;

```

```

2: Version 2
Bubble Sort (Array A[])
For i = 1 to n - 1
For j = 1 to n - i
If (A[j] > A[j+1]) then
Swap(A[j], A[j+1])

```

Compiled: November 2018 Last update: January 2023 Page 54

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Analysis of the Bubble Sort Algorithm

The innermost statement, the if, takes $O(1)$ time. It doesn't necessarily take the same time when the condition is true as it does when it is false, but both times are bounded by a constant.

The outer loop executes n times, but the inner loop executes a number of times that depends on i .

The first time the inner for executes, it runs $i = n-1$ times. The second time it runs $n-2$ times as shown in the table below.

The total number of times the inner if statement executes is therefore:

$$(n-1) + (n-2) + \dots + 3 + 2 + 1$$

This is the sum of an arithmetic series.

The value of the sum is $n(n-1)/2$.

So the running time of bubble sort is $O(n(n-1)/2)$, which is $O((n^2-n)/2)$.

Using the rules for big-O given earlier, this bound simplifies to $O((n^2)/2)$ by ignoring a smaller term, and to $O(n^2)$, by ignoring a constant factor.

Thus, bubble sort is an $O(n^2)$ algorithm.

Compiled: November 2018 Last update: January 2023 Page 55

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

ii) Selection Sort Algorithm

The selection sort improves on the bubble sort by making only one exchange for every pass through the list.

□ In order to do this, a selection sort looks for the largest value as it makes a pass and, after completing the pass, places it in the proper location. As

with a bubble sort, after the first pass, the largest item is in the correct place.

□ After the second pass, the next largest is in place. This process continues and requires $n-1$ passes to sort n items, since the final item must be in place after the $(n-1)$ st pass.

The Selection sort performs sorting by searching for the minimum value number and placing it into the first or last position according to the order (ascending or descending). The process of searching the minimum key and placing it in the proper position is continued until the all the elements are placed at right position.

The basic idea of selection sort:

- Consider the positions in the array from left to right
- For each position, find the element that belongs there and put it in place by swapping it with the element that's currently there

The figure shows the entire sorting process. On each pass, the largest remaining item is selected and then placed in its proper location. The first pass places 93, the second pass places 77, the third places 55, and so on.

Compiled: November 2018 Last update: January 2023 Page 56

CSC315 - Algorithms & Complexity Analysis - Orhionkpaiyo, B. C

The Algorithm

SelectionSort

Input: An array $A[1::n]$ of n elements.

Output: $A[1::n]$ sorted in non-decreasing order.

1. for $i \leftarrow 1$ to $n-1$
2. $k \leftarrow i$
3. for $j \leftarrow i+1$ to n //Find the i th smallest element
4. if $A[j] < A[k]$ then $k \leftarrow j$
5. end for
6. if $k \neq i$ then interchange $A[i]$ and $A[k]$
7. end for

Analysis of Selection sort algorithm

To sort n elements, selection sort performs $n - 1$ passes:

Compiled: November 2018 Last update: January 2023 Page 57

CSC315 - Algorithms & Complexity Analysis - Orhionkpaiyo, B. C

- on 1st pass, it performs $n - 1$ comparisons
- on 2nd pass, it performs $n - 2$ comparisons
- on the $(n-1)$ st pass, it performs 1 comparison

Adding up the comparisons for each pass, we get:

$$C(n) = (n - 1) + (n - 2) + \dots + 2 + 1$$

The resulting formula for $C(n)$ is the sum of arithmetic sequence

$$C(n) = (n - 1) + (n - 2) + \dots + 2 + 1 =$$

Formula of the sum of this type of arithmetic sequence is:

Thus, we can simplify our expression for $C(n)$ as follows:

$$C(n) =$$

$$= n^2/2 - n/2$$

In characterizing the time complexity of an algorithm, we'll focus on the largest term in its operation-count expression

$$C(n) = n^2/2 - n/2 \approx n^2/2$$

In addition, we'll typically ignore the coefficient of the largest term

$$\text{Thus } n^2/2 \approx n^2$$

We specify the largest term using big-O notation.

$$C(n) = n^2/2 - n/2 = O(n^2)$$

Compiled: November 2018 Last update: January 2023 Page 58

CSC315 - Algorithms & Complexity Analysis - Orhionkpaiyo, B. C

Thus the time complexity of selection sort algorithm is $O(n^2)$

That is, selection sort algorithm performs $O(n^2)$ comparisons

We can also compute the number of moves performed by selection sort algorithm.

After each of the $n-1$ passes to find the smallest remaining element, the algorithm performs a swap to put the element in place.

- $n-1$ swaps, 3 moves per swap
- $M(n) = 3(n-1) = 3n-3$

Thus selection sort performs $O(n)$ moves.

iii). Insertion Sort Algorithm

Insertion Sort iterates through a list of records. Each record is inserted in turn at the correct position within a sorted list composed of those records already processed.

□ Insertion sort uses two sets of arrays where one stores the sorted data and other on unsorted data.

The primary concept behind insertion sort is to insert each item into its appropriate place in the final list.

□ One way to think about this is to imagine that you have a stack of phone bills from the past two years and that you wish to organize them by date.

o A fairly natural way to do this might be to look at the first two bills and put them in order. Then take the third bill and put it into the right order with respect to the first two, and so on. As you take

Compiled: November 2018 Last update: January 2023 Page 59

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

each bill, you would add it to the sorted pile that you have already made.

Summary of operations of Insertion Sort

- Take multiple passes over the array
- Keep already sorted array at low-end
- Find next unsorted element
- Insert it in correct place, relative to the ones already sorted

Below is an illustration of Insertion Sort. Each column shows the array after the iteration with the indicated value of i in the outer for loop. Values above the line in each column have been sorted. Arrows indicate the upward motions of records through the array.

The Algorithm

Algorithm insertionSort(A, n)

Input: array A of n integers

Output: the sorted array A

1. for $i \leftarrow 1$ to $n - 1$ do
2. $x \leftarrow A[i]$
3. for $j \leftarrow i - 1$ downto 0 do
4. if $x < A[j]$ then $A[j+1] \leftarrow A[j]$
5. else break
6. $A[j+1] \leftarrow x$
7. return A

Compiled: November 2018 Last update: January 2023 Page 60

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Time complexity Analysis of Insertion Sort

By inspecting the pseudo code, we can determine the maximum number of primitive operations executed by an algorithm, as a function of the input size.

Algorithm insertionSort(A, n) operations total # operations

Input array A of n integers

Output the sorted array A

for $i \leftarrow 1$ to $n - 1$ do 0 0

$x \leftarrow A[i]$ 1 $n - 1$

for $j \leftarrow i - 1$ downto 0 do 0 0

if $x < A[j]$ then $A[j+1] \leftarrow (n -$

```

A[j] 2 1)n
else break 0 0
A[j+1] ← x 1 n - 1
return A 1 1

```

Total $n^2 + n - 1$

There are $n-1$ passes to sort n items. The iteration starts at position 1 and moves through position $n-1$. The maximum number of comparisons for an insertion sort is the sum of the first $n-1$

Total number of comparisons i. e. $C(n) = 1 + 2 + \dots + (n - 1) =$

Compiled: November 2018 Last update: January 2023 Page 61

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

4. Algorithm Design Techniques/Strategies

An algorithm design technique (or “strategy” or “paradigm”) is a general approach to solving problems algorithmically that is applicable to a variety of problems from different areas of computing.

4.1 The Greedy Algorithm Design Technique

The greedy approach is one of the common algorithm design techniques.

□ Greedy algorithms are usually designed to solve optimization problems in which a quantity is to be minimized or maximized.

The greedy method is a simple strategy of progressively building up a solution, one element at a time, by choosing the best possible element at each stage.

- o At each stage, a decision is made regarding whether or not a particular input is in an optimal solution. Decisions are never reversed.

This is done by considering the inputs in an order determined by some selection procedure.

□ If the inclusion of the next input, into the partially constructed optimal solution will result in an infeasible solution then this input is not added to the partial solution.

The selection procedure itself is based on some optimization measure. Several optimization measures are plausible for a given problem. Most of them, however, will result in algorithms that generate sub-optimal solutions.

On each step—and this is the central point of this technique—the choice made must be:

- o feasible, i.e., it has to satisfy the problem’s constraints
- o locally optimal, i.e., it has to be the best local choice among all feasible choices available on that step

Compiled: November 2018 Last update: January 2023 Page 62

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

o irrevocable, i.e., once made, it cannot be changed on subsequent steps of the algorithm

For the greedy strategy to work well, it is necessary that the problem under consideration has two characteristics:

□ Greedy choice property: A global optimum can be arrived at by selecting a local optimum

□ Optimal substructure: A problem exhibits optimal substructure if an optimal solution to the problem contains within it optimal solutions to sub-problems.

o i.e. an optimal solution to the problem contains optimal solution to the sub problem

o This implies we can solve sub-problems and build up the solutions to solve larger problems. If an optimal solution A to S begins with activity 1, then $A' = A - \{1\}$ is optimal to $S' = \{i \in S: si \neq f1\}$

To apply the greedy technique to a problem, we must take into consideration the following:

- i. A local criteria to allow the selection,
- ii. A condition to determine if a partial solution can be completed,
- iii. A procedure to test that we have the optimal solution.

4.1.1 Applications of Greedy Algorithms

Greedy algorithms can be applied in wide range of problems which include the following problems:

Compiled: November 2018 Last update: January 2023 Page 63

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

- o Activity selection problem
- o Minimum spanning tree
- o Knapsack problem
- o Unit task scheduling with deadline

However, its applications are not limited to these problem areas.

1. Activity Selection Problem

Problem: Schedule an exclusive resource in competition with other entities.

Examples include:

- o Scheduling a hall for lectures (only one course can hold at a time) when several groups want to use it.
- o Renting out some piece of equipment to different people is another example.

o Others include requests for boats to use a repair facility while they are in port, and planning weekend schedule.

The activity-selection problem is the problem of selecting the largest set of mutually compatible activities i.e. finding a maximum set of non-overlapping tasks (each with a fixed start and finish time) that can be completed by a single processor

Suppose we are given a set of n lectures, and lecture i starts at time s_i and ends at time f_i . The goal is to use the minimum number classrooms to schedule all lectures so that no two occur at the same time in the same room. As an illustration of the problem, consider the sample instance

Compiled: November 2018 Last update: January 2023 Page 64

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

(a) An instance of the lectures (a through j).

(b) A solution in which all lectures are scheduled using 3 classrooms: each row represents a set of lectures that can all be scheduled in a single classroom.

The diagram below helps to visualize this (b) by incorporating time.

Problem Description: There is a set of activities $S = \{a_1, a_2, \dots, a_n\}$. Each activity a_i has a start time s_i and a finish time f_i , where $f_i > s_i \geq 0$. The duration of activity a_i is (s_i, f_i) . Two activities a_i and a_j are compatible if $s_i \geq f_j$ or $s_j \geq f_i$. Two activities a_i and a_j overlap if $[s_i, f_i] \cap [s_j, f_j] \neq \emptyset$. The objective is to select a maximum number of pair wise non-overlapping activities (i.e. select a subset of S that has maximum number of activities that are mutually compatible).

Compiled: November 2018 Last update: January 2023 Page 65

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Consider a set of requests for a room. Only one person can reserve the room at a time, and you want to allow the maximum number of requests.

The requests for periods (s_i, f_i) are:

$(1,4), (3,5), (0,6), (5,7), (3,8), (5,9), (6,10), (8,11), (8,12), (2,13), (12,14)$

Which ones should we schedule?

Different approaches for selecting activities/jobs:

- i. Earliest start time: Considers jobs in ascending order of start times s_i
- ii. Shortest request time: Considers jobs in ascending order of interval length $f_i - s_i$
- iii. Fewest conflicting jobs: For each job, count the number of conflicting jobs c_i . Schedule in ascending order of conflicts c_i .

iv. Earliest finish time: Considers jobs in ascending order of start time f_i .

This is mostly used because it yields optimal choice. It leaves the

resource available for as many other activities as possible.

Example:

i a1 a2 a3 a4 a5 a6 a7 a8 a9 a10 a11

si 1 3 0 5 3 5 6 8 8 2 12

fi 4 5 6 8 8 9 10 11 12 13 14

Possible sets of mutually compatible activities:

i. {a3, a9, a11}

ii. {a1, a4, a8, a11}

iii. {a2, a4, a9, a11}

Observe that ii and iii have the largest subsets of compatible activities

Exercise: Using the above table, determine the possible sets of mutually compatible activities by applying the first three approaches above.

Compiled: November 2018 Last update: January 2023 Page 66

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Points to remember

□ For this algorithm we have a list of activities with their starting time and finishing time.

□ Our goal is to select maximum number of non-conflicting activities that can be performed by a person or a machine, assuming that the person or machine involved can work on a single activity at a time.

□ Any two activities are said to be non-conflicting if starting time of one activity is greater than or equal to the finishing time of the other activity.

□ In order to solve this problem we first sort the activities as per their finishing time in ascending order.

□ Then we select non-conflicting activities.

Problem

Consider the following 8 activities with their starting and finishing time.

Activity a1 a2 a3 a4 a5 a6 a7 a8

start 1 0 1 4 2 5 3 4

finish 3 4 2 6 9 8 5 5

Our goal is to find non-conflicting activities.

For this we follow the given steps

1. sort the activities as per finishing time in ascending order
2. select the first activity
3. select the new activity if its starting time is greater than or equal to the previously selected activity

REPEAT step 3 till all activities are checked

Step 1: sort the activities as per finishing time in ascending order

Compiled: November 2018 Last update: January 2023 Page 67

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Sorted Activity a3 a1 a2 a7 a8 a4 a6 a5

start 1 1 0 3 4 4 5 2

finish 2 3 4 5 5 6 8 9

Step 2: select the first activity

Step 3: select next activity whose start time is greater than or equal to the finish time of the previously selected activity

Pseudo Code

Set $i = 0$; //pointing at first element

for $j = 1$ to $n-1$ do

if start time of $j \geq$ finish time of i then

Print j

Set $i = j$

endif

endfor

The Greedy Activity Selector Algorithm:

Greedy_activity_selector (s, f)

1. Sort input activities in order of increasing finishing time ($f_1 \leq f_2 \leq \dots \leq f_n$)
2. $n \leftarrow \text{length}[s]$
3. $A \leftarrow \{i\}$
4. $j \leftarrow 1$
5. for $i \leftarrow 2$ to n
6. if $s_i \geq f_j$ then
7. $A \leftarrow A \cup \{i\}$
8. $j \leftarrow i$
9. return A

Compiled: November 2018 Last update: January 2023 Page 68

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The set A collects the selected activities. The variable j specifies the most recent addition to A . Since the activities are considered in order of non-decreasing finishing time, f_j is always the maximum finishing time of any activity in A .

Analysis of the Algorithm

Running time of line 1 in the above depends on the sort algorithm used.

With merge sort algorithm, it requires $O(n \log n)$ time.

$O(n)$ for the greedy collection of activities (i.e The algorithm takes as input a set of activities S (each activity stored as a pair of numbers (s_i, f_i)) and the total number of activities n).

Total running time is $O(n \log n) + O(n) = O(n \log n)$

2. Minimum Spanning Tree

A spanning tree of a graph $G = (V, E)$ is a tree that contains all vertices of V and is a sub-graph of G . A single graph can have multiple spanning trees.

A Tree is a connected undirected graph that contains no cycles.

A Cycle is a sequence of nodes and edges without repetitions other than the starting and ending nodes. Example

Compiled: November 2018 Last update: January 2023 Page 69

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

An Undirected Graph is a graph where the edges have no directions. An undirected graph is connected when there is a path between every pair of vertices. In a connected graph, there are no unreachable vertices

2 2

1 1

3 3

(a) Undirected graph (b) Directed graph

Lemma 1: Let T be a spanning tree of a graph G . Then

1. Any two vertices in T are connected by a unique simple path.
2. If any edge is removed from T , then T becomes disconnected.
3. If we add any edge into T , then the new graph will contain a cycle.
4. Number of edges in T is $n-1$.

A spanning tree for a connected graph is a tree whose vertex set is the same as the vertex set of the given graph, and whose edge set is a subset of the edge set of the given graph. i.e., any connected graph will have a spanning tree.

Weight of a spanning tree $w(T)$ is the sum of weights of all edges in T . The Minimum spanning tree (MST) is a spanning tree with the smallest possible weight.

The following diagrams illustrate a graph and the possible spanning trees from the graph.

Compiled: November 2018 Last update: January 2023 Page 70

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

A greedy method to obtain the minimum spanning tree would construct the tree edge by edge, where each edge is chosen accounting to some optimization criterion. An obvious criterion would be to choose an edge which adds a minimum weight to the total weight of the edges selected so far. There are two ways in which this criterion can be achieved.

- i. The set of edges selected so far always forms a tree, the next edge to be added is such that not only it adds a minimum weight, but also forms a

tree with the previous edges; it can be shown that the algorithm results in a minimum cost tree; this algorithm is called Prim's algorithm.

ii. The edges are considered in non decreasing order of weight; the set T of edges at each stage is such that it is possible to complete T into a tree; thus T may not be a tree at all stages of the algorithm; this also results in a minimum cost tree; this algorithm is called Kruskal's algorithm.

i). Prim's algorithm.

The algorithm grows the spanning tree starting from an arbitrary vertex. Let $G = (V, E)$, where for simplicity V is taken to be the set of integers $\{1, 2, \dots, n\}$. The algorithm begins by creating two sets of vertices: $X = \{1\}$ and $Y = \{2, 3, \dots, n\}$. It then grows a spanning tree, one edge at a time. At each step, it finds an edge $(x,$

Compiled: November 2018 Last update: January 2023 Page 71

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

$y)$ of minimum weight, where $x \in X$ and $y \in Y$ and moves y from Y to X . This edge is added to the current minimum spanning tree edges in T . This step is repeated until Y becomes empty.

The main idea of Prim's algorithm:

- Start with one (any) vertex.
- Branch outwards to grow your connected component.
- Consider only edges that leave the connected component.
- Add smallest considered edge to your connected component.
- Continue until a spanning tree is created

Steps for finding MST using Prim's Algorithm:

1. Create MST set that keeps track of vertices already included in MST.
2. Assign key values to all vertices in the input graph. Initialize all key values as INFINITE (∞). Assign key values like 0 for the first vertex so that it is picked first.
3. While MST set doesn't include all vertices.
 1. Pick vertex u which is not in MST set and has minimum key value. Include ' u ' to MST set.
 2. Update the key value of all adjacent vertices of u . To update, iterate through all adjacent vertices. For every adjacent vertex v , if the weight of edge $u.v$ less than the previous key value of v , update key value as a weight of $u.v$.

Simply put, the implementation of Prim's Algorithm is explained in the following steps-

Step1:

- Randomly choose any vertex.
- The vertex connecting to the edge having least weight is usually selected.

Compiled: November 2018 Last update: January 2023 Page 72

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Step2:

- Find all the edges that connect the tree to new vertices.
- Find the least weight edge among those edges and include it in the existing tree.
- If including that edge creates a cycle, then reject that edge and look for the next least weight edge.
- Keep repeating this step 2 until all the vertices is included and Minimum Spanning Tree (MST) is obtained.

The following diagrams illustrate the operation of Prim's algorithm

Step 1: Select edge 1- 6

Compiled: November 2018 Last update: January 2023 Page 73

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Step 2: Add edge 6 - 5

Step 3: Add edge 5 - 4

Step 4: Add edge 4 - 3

Step 5: Add edge 3 - 2

Compiled: November 2018 Last update: January 2023 Page 74

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Step 6: Add edge 2 - 7

The cost of the MST is as shown in the following table

Selected Edges Weight

1-6 10

6-5 25

5-4 22

4-3 12

3-2 16

2-7 14

Total Weight 99

Consider another example

Compiled: November 2018 Last update: January 2023 Page 75

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The final generated MST is as shown below

Selected Edges Weight

1-7 1

7-2 4

7-3 9

3-4 3

4-5 17

1-6 23

Total Weight 57

Compiled: November 2018 Last update: January 2023 Page 76

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The Algorithm

$\{N[y]$ is neighbour of $y\}$

$\{C[y]$ is cost of edge connecting $y\}$

$\{\text{Vertex } y \text{ not adjacent to } 1\}$

Time complexity Analysis of the Algorithm

The time complexity of the algorithm is computed as follows:

The for loop in Step 2 requires $O(n)$ time.

The time taken by steps 3 - 6 is $O(n)$.

The time taken by Step 9 to search for a vertex y closest to X is $O(n)$ per iteration. This is because the algorithm inspects each entry in the vector representing the set Y . Since this step is executed $n - 1$ times, the overall time taken by Step 9 is $O(n^2)$.

Steps 10 - 13 cost $O(1)$ time per iteration for a total of $O(n)$ time.

The for loop in Step 14 is executed $2m$ times, where $m = |E|$. This is because each edge (y, w) is inspected twice: once when y is moved to X and the other when w is moved to X . Hence, the overall time required by Step 14 is $O(m)$.

The test in Step 15 is executed exactly m times, and Steps 16 and 17 are executed at most m times. Thus, Steps 14 to 17 cost $O(m)$ time.

Compiled: November 2018 Last update: January 2023 Page 77

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

It follows that the time complexity of the algorithm is $O(m + n^2) = O(n^2)$.

However, using a heap improves the algorithm to $O(m \log n)$ time

ii). Kruskal's Algorithm

Kruskal's algorithm works by maintaining a forest consisting of several spanning trees that are gradually merged until finally the forest consists of exactly one tree: a minimum cost spanning tree.

Kruskal's algorithm is summarized as follows:

- Sort the edges in graph G by non-decreasing weight.
- For each edge in the sorted list, include that edge in the spanning tree T if it does not form a cycle with the edges currently included in T ; otherwise discard it.

Consider the weighted graph below. The first edge that is added is (1, 2), followed by edges (1, 3), (4, 6), and (5, 6). Notice that edge (2, 3) creates a cycle as shown in figure f, and hence is discarded. For the same reason edge (4, 5) is also discarded, as shown in figure g. Finally, edge (3, 4) is included, which results in the minimum spanning tree (V, T) shown in fig. (h).

Compiled: November 2018 Last update: January 2023 Page 78

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The Algorithm

Time complexity

We analyze the time complexity of the algorithm as follows:

Steps 1 and 2 cost $O(m \log m)$ and $\Theta(n)$, respectively, where $m = |E|$.

Step 6 costs $\Theta(1)$, and since it is executed at most m times, its total cost is $O(m)$.

Step 7 is executed exactly $n - 1$ times for a total of $\Theta(n)$ time.

The union operation is executed $n - 1$ times

The find operation is executed at most $2m$ times.

The overall cost of these two operations (i.e. union and find) is $O(m \log^* n)$.

Compiled: November 2018 Last update: January 2023 Page 79

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Thus, the overall running time of the algorithm is dominated by the sorting step, i.e., $O(m \log m)$.

iii). The Knapsack Problem

There are two variants of the knapsack problem:

- Integral or 0-1 Knapsack problem,
- Fractional knapsack problem

The 0-1 knapsack problem is posed as follows. A thief robbing a store finds n items; the i th item is worth v_i dollars and weighs w_i pounds, where v_i and w_i are integers. He wants to take as valuable a load as possible, but he can carry at most W pounds in his knapsack for some integer W . Which items should he take? (This is called the 0-1 knapsack problem because each item must either be taken or left behind; the thief cannot take a fractional amount of an item or take an item more than once e.g. TV set)

In the fractional knapsack problem, the setup is the same, but the thief can take fractions of items, rather than having to make a binary (0-1) choice for each item. You can think of an item in the 0-1 knapsack problem as being like a gold ingot (mass of metal), while an item in the fractional knapsack problem is more like gold dust.

Both knapsack problems exhibit the optimal-substructure property.

o For the 0-1 problem, consider the most valuable load that weighs at most W pounds. If we remove item j from this load, the remaining load must be the most valuable load weighing at most $W - w_j$ that the thief can take from the $n - 1$ original items excluding j .

Compiled: November 2018 Last update: January 2023 Page 80

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

o For the comparable fractional problem, consider that if we remove a weight w of one item j from the optimal load, the remaining load must be the most valuable load weighing at most $W - w$ that the thief can take from the $n - 1$ original items plus $w_j - w$ pounds of item j .

However, only the fractional knapsack problem has the greedy choice property. Thus, the fractional knapsack problem is solvable by a greedy strategy, whereas the 0-1 problem is not (but solvable by dynamic programming). Therefore, our focus is the fractional knapsack problem.

o The Fractional Knapsack Problem

The fractional knapsack problem is defined as follows:

Given a list of n objects say $\{I_1, I_2, \dots, I_n\}$ and a knapsack (or bag) with capacity, M . Each object I_i has a weight W_i and a profit of P_i . If a fraction x_i (where $x_i \in \{0, \dots, 1\}$) of an object I_i is placed into a knapsack, then a profit of $p_i x_i$ is earned.

The problem (or objective) is to fill a knapsack up to its maximum capacity M , which maximizes the total profit earned.

Mathematically:

Where p_i and w_i are the profit and weight of i th object and x_i is the fraction of i th object to be selected.

Compiled: November 2018 Last update: January 2023 Page 81

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Example:

Given $n = 3$, $(p_1, p_2, p_3) = \{25, 24, 15\}$

$(w_1, w_2, w_3) = \{18, 15, 10\}$ $M = 20$

Solution

Some of the feasible solutions are shown in the following table.

Solution No x_1 x_2 x_3 $\sum w_i x_i$ $\sum p_i x_i$

1 1 2/15 0 20 28.2

2 0 2/3 1 20 31.0

3 0 1 1/2 20 31.5

These solutions are obtained by different greedy strategies.

Greedy strategy I: In this case, the items are arranged by their profit values. Here the item with maximum profit is selected first. If the weight of the object is less than the remaining capacity of the knapsack then the object is selected full and the profit associated with the object is added to the total profit. Otherwise, a fraction of the object is selected so that the knapsack can be filled exactly. This process continues from selecting the highest profitable object to the lowest profitable object till the knapsack is exactly full.

Thus, the first item has the maximum profit (25) and is selected first. After filling the item ($w = 18$), the sack is left with only 2 capacity (i.e. $20 - 18$). Next, a fraction of the second item (profit = 24) is selected. Since the weight, 15, exceeds the maximum, a fraction of it, $\frac{2}{15}$, is taken. Now the knapsack is full (i.e. 20) so 3rd item is not selected. Hence, the total profit = $25 + 2 \times \frac{24}{15} = 28.2$ units and the solution set is $(x_1, x_2, x_3) = (1, \frac{2}{15}, 0)$.

Compiled: November 2018 Last update: January 2023 Page 82

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Greedy strategy II: In this case, the items are arranged by fair weights. Here the item with minimum weight is selected first and the process continues like greedy strategy-I till the knapsack is exactly full.

Thus, 3rd, and 2nd items are selected. After selecting the 3rd item, the sack remains 10 to be filled, so only a fraction ($\frac{10}{15}$) of the 2nd item is taken. That is $\frac{10}{15} \times 24 = 16$. Hence, the profit is $25 + 16 = 41$.

Greedy strategy III: In this case, the items are arranged by profit/weight ratio and the item with maximum profit/weight ratio is selected first and the process continues like greedy strategy-I till the knapsack is exactly full.

Using this approach, select those objects with maximum value of $\frac{p_i}{w_i}$, that is, select those objects first with maximum profit per unit weight. Since $\frac{p_1}{w_1} = 1.3$, $\frac{p_2}{w_2} = 1.6$, $\frac{p_3}{w_3} = 1.5$ respectively, select 2nd object first, then 3rd object followed by 1st object. The total profit (i.e. 31.5 (i.e. $24 + 7.5$)) and the solution set $(x_1, x_2, x_3) = (0, 1, \frac{5}{15})$ i.e. total weight selected from $x_1, x_2, x_3 = 0, 15, 5 = 20$.

Therefore, it is clear from the above strategies that the Greedy method generates optimal solution if we select the objects with respect to their profit to weight ratios.

Compiled: November 2018 Last update: January 2023 Page 83

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The Algorithm

Time Complexity Analysis

Sorting of n items in decreasing order of the ratio takes $O(n \log n)$ time (which is the most efficient runtime possible for sort algorithms)

Line 6 (i.e. the while loop) takes $O(n)$ time

Therefore, total time is $O(n \log n)$

iv). Unit Task Scheduling with Deadlines

A unit-time task is a job, such as a program to be run on a computer that requires exactly one unit of time to complete. Given a finite set S of unit-time tasks, a schedule for S is a permutation of S specifying the order in which to perform these tasks.

o The first task in the schedule begins at time 0 and finishes at time 1, the second task begins at time 1 and finishes at time 2, and so on.

Compiled: November 2018 Last update: January 2023 Page 84

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

o Because each job takes the same amount of time, we will think of a Schedule S as consisting of a sequence of job “slots” $1, 2, 3, \dots$ where $S(t)$ is the job scheduled in slot, t

The problem of scheduling unit-time tasks with deadlines and penalties for a single processor has the following inputs:

- a set $S = \{a_1, a_2, \dots, a_n\}$ of n unit-time tasks;
- a set of n integer deadlines $d_1, d_2, \dots, d_n$, such that each d_i satisfies $1 \leq d_i \leq n$ and task a_i is supposed to finish by time d_i ; and
- a set of n nonnegative weights or penalties $w_1, w_2, \dots, w_n$, such that we incur a penalty of w_i if task a_i is not finished by time d_i , and we incur no penalty if a task finishes by its deadline.

We wish to find a schedule for S that minimizes the total penalty incurred for missed deadlines.

Task Scheduling Problem:

Given is a set S of n tasks $\{a_1, \dots, a_n\}$ and a single processor. Each task takes a unit time to execute and tasks have deadlines $d_1, \dots, d_n$. Tasks have non-negative penalties $w_1, \dots, w_n$ if miss the deadline.

Goal: Find a schedule S where the total penalty of the tasks that miss their deadline is minimized – i.e., the total penalty of the tasks that meet their deadline is maximized

NOTE:

- o A feasible solution is a subset of jobs J such that each job is completed by its deadline.
- o An optimal solution is a feasible solution with maximum profit value.

Compiled: November 2018 Last update: January 2023 Page 85

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

A Greedy Strategy for Unit Task Scheduling:

- i. Sort all the given jobs in decreasing order of their profit.
- ii. Check the value of maximum deadline and draw a Gantt chart where maximum time on Gantt chart is the value of maximum deadline.
- iii. Pick up the jobs one by one on the Gantt chart as far as possible from 0 ensuring that the job gets completed before its deadline.

Given the following six jobs, their deadlines and associated profits, we illustrate the above strategy as follows:

Jobs J1 J2 J3 J4 J5 J6

Deadlines 5 3 3 2 4 2

Profits 20 18 19 30 12 10

Step 1: Sort all the given jobs in decreasing order of their profit

Jobs J4 J1 J3 J2 J5 J6

Deadlines 2 5 3 3 4 2

Profits 30 20 19 18 12 10

Step 2:

Value of maximum deadline = 5.

So, draw a Gantt chart with maximum time on Gantt chart = 5 units as shown

Gantt chart

Compiled: November 2018 Last update: January 2023 Page 86

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Step 3:

1. First, take job J4 since its deadline is 2, so we place it in the first empty cell before deadline 2
2. Then take job J1 since its deadline is 5, so we place it in the first empty cell before deadline 5 as-
3. Next, take job J3 since its deadline is 3, so we place it in the first empty cell before deadline 3
4. Next, take job J2. Since its deadline is 3, so we place it in the first empty cell before deadline 3. But the second and third cells are already filled, so we place job J2 in the first cell
5. Next, take job J5. Since its deadline is 4, so we place it in the first empty cell before deadline 4

NOTE: The only job left is J6 whose deadline is 2. All the slots before deadline 2 are already occupied therefore J6 cannot be completed.

Thus the optimal schedule of the jobs is: J2, J4, J3, J5, J1

Total profit: $30 + 20 + 19 + 18 + 12 = 99$

Consider another example:

Using greedy algorithm, find an optimal schedule for the following jobs with $n = 6$.

Profits: (P1, P2, P3, P4, P5, P6) = (20, 15, 10, 7, 5, 3) and

Deadline (d1, d2, d3, d4, d5, d6) = (3, 1, 1, 3, 1, 3)

Compiled: November 2018 Last update: January 2023 Page 87

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Solution:

Job i 1 2 3 4 5 6

Profit gi 20 15 10 7 5 3

Deadline di 3 1 1 3 1 3

Steps:

i. Here jobs are already sorted in decreasing order of their profit

Let $P = \min(6, 3) = 3$

F = 0 1 2 3

Job 0 0 0 0

selected

ii. d1 = 3: assign job 1 to position 3

F = 0 1 2 0

Job 0 0 0 1

selected

iii. d2 = 1: assign job 2 to position 1

F = 0 0 2 0

Job 0 2 0 1

selected

iv. d3 = 1: No free position, so reject the job

v. d4 = 3: assign job 4 to position 2 as position 3 is not free, but position 2 is free

F = 0 0 0 0

Job 0 2 4 1

selected

Compiled: November 2018 Last update: January 2023 Page 88

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

vi. Now, no more free position is left so no more jobs can be scheduled

The final optimal sequence: Execute the job order 2, 4, 1 with total profit value 42

The Algorithm

1:

2:

Sort all the n jobs in decreasing order of their profit

for $i = 1$ to n do

Set $k = \min(d_{\max}, \text{DEADLINE}(i))$ //where

$\text{DEADLINE}(i)$ denotes

deadline of i th job

while $k \geq 1$ do

if timeslot[k] is EMPTY then

timeslot[k] = job(i)

break

endif

Set $k = k - 1$

endwhile

endfor

Compiled: November 2018 Last update: January 2023 Page 89

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Analysis of the algorithm

Sort task according to decreasing order of deadline = $O(n \log n)$

for each task find slot in array of size $n = O(n^2)$

Total time = $O(n \log n) + O(n^2) = O(n^2)$

Compiled: November 2018 Last update: January 2023 Page 90

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

5. Divide-and-Conquer Algorithm Design Strategy

Many useful algorithms are recursive in structure: to solve a given problem, they call themselves recursively one or more times to deal with closely related sub-problems.

These algorithms typically follow a divide-and-conquer approach:

- o they break the problem into several sub-problems that are similar to the original problem but smaller in size,
- o solve the sub-problems recursively, and
- o then combine these solutions to create a solution to the original problem.

The divide-and-conquer paradigm involves three steps at each level of the recursion:

Divide the problem into a number of sub-problems.

Conquer the sub-problems by solving them recursively. If the sub-problem sizes are small enough, however, just solve the sub-problems in a straightforward manner.

Combine the solutions to the sub-problems into the solution for the original

problem.

When an algorithm contains a recursive call to itself, its running time can often be described by a recurrence. Recurrence is briefly explained in the following section.

5.1 Recurrences

A recurrence is a recursive description of a function, or in other words, a description of a function in terms of itself.

- o Each term of the sequence is defined as a function of the preceding terms.

Compiled: November 2018 Last update: January 2023 Page 91

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Like all recursive structures, a recurrence consists of one or more base cases and one or more recursive cases. Each of these cases is an equation or inequality, with some function value $f(n)$ on the left side.

- o The base cases give explicit values for a (typically finite, typically small) subset of the possible values of n .
- o The recursive cases relate the function value $f(n)$ to function value $f(k)$ for one or more integers $k < n$;
- o typically, each recursive case applies to an infinite number of possible values of n .

For example, the following recurrence (written in two different but standard ways) describes the identity function $f(n) = n$:

In both presentations, the first line is the only base case, and the second line is the only recursive case.

Recurrences arise naturally in the analysis of algorithms, especially recursive algorithms. In many cases, we can express the running time of an algorithm as a recurrence, where the recursive cases of the recurrence correspond exactly to the recursive cases of the algorithm.

The recurrence: $T(n) = aT(n/b) + f(n)$

is called the general divide-and-conquer recurrence and describes the running time of an algorithm that divides a problem of size n into a sub-problems, each of size n/b , where a and b are positive constants. The ' a ' sub-problems are solved recursively, each in $T(n/b)$.

- o The function encompasses the cost of dividing the problem and combining the results of the sub-problems.

Compiled: November 2018 Last update: January 2023 Page 92

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

The Master Theorem:

The Master theorem provides a “cookbook” method for solving recurrences. It may be applied whenever appropriate, rather than re-deriving the solution for the recurrence. To use the master method, we simply determine which case (if any) of the master theorem applies

For any recurrence relation of the form

$$T(n) = aT(n/b) + cn^k, T(1) = c,$$

The following relationship may hold:

Where $a, b \geq 1$, $k \geq 0$, and $c > 0$ are constants

The above relation also appear in some text as:

$$f(n) = af(n/b) + cn^k$$

We can apply the theorem to solve problems as follows

Example 1:

$$T(n) = 3T(n/5) + 8n^2$$

Because $a = 3$, $b = 5$, $c = 8$, and $k = 2$, we find that $3 < 5^2$.

Applying case (3) of the theorem, $T(n) = \Theta(n^2)$.

Example 2:

$$T(n) = 2T(n/2) + n, T(1) = 1$$

Because $a = 2$, $b = 2$, $c = 1$, and $k = 1$, we find that $2 = 2^1$.

Applying case (2) of the theorem, $T(n) = \Theta(n \log n)$.

Compiled: November 2018 Last update: January 2023 Page 93

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

5.2 Quicksort Algorithm

The quicksort algorithm is a sorting algorithm that sorts a collection by choosing a pivot point, and partitioning the collection around the pivot, so that elements smaller than the pivot are before it, and elements larger than the pivot are after it.

ALGORITHM Quicksort($A[l..r]$)

//Sorts a sub-array by quicksort

Input: Sub-array of array $A[0..n - 1]$, defined by its left and right indices l and r

Output: Sub-array $A[l..r]$ sorted in nondecreasing order

if $l < r$

$s \leftarrow \text{Partition}(A[l..r])$ //s is a split position

Quicksort($A[l..s - 1]$)

Quicksort($A[s + 1..r]$)

Partition: The performance of quicksort depends heavily on the performance of the split operation. The effect of splitting from l to r is:

- $x = A[l]$ is moved to its correct location at $A[m]$;
- no item in $A[l..m-1]$ is larger than x ;

- no item in $A[m+1..r]$ is smaller than x .

ALGORITHM HOARE-PARTITION(A, p, r)

```

1.  $x = A[p]$ 
2.  $i = p - 1$ 
3.  $j = r + 1$ 
4. while true
5. repeat
6.  $j = j - 1$ 
7. until  $A[j] \leq x$ 
8. repeat
9.  $i = i + 1$ 

```

Compiled: November 2018 Last update: January 2023 Page 94

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

```

10. until  $A[i] \geq x$ 
11. if  $i < j$ 
12. exchange  $A[i]$  with  $A[j]$ 
13. else return  $j$ 

```

The left-to-right scan, denoted below by index pointer i , starts with the second element. Since we want elements smaller than the pivot to be in the left part of the sub-array, this scan skips over elements that are smaller than the pivot and stops upon encountering the first element greater than or equal to the pivot.

The right-to-left scan, denoted below by index pointer j , starts with the last element of the sub-array. Since we want elements larger than the pivot to be in the right part of the sub-array, this scan skips over elements that are larger than the pivot and stops on encountering the first element smaller than or equal to the pivot.

After both scans stop, three situations may arise, depending on whether or not the scanning indices have crossed. If scanning indices i and j have not crossed, i.e. $i < j$, we simply exchange $A[i]$ and $A[j]$ and resume the scans by incrementing i and decrementing j , respectively:

If the scanning indices have crossed over, i.e., $i > j$, we will have partitioned the sub-array after exchanging the pivot with $A[j]$:

Finally, if the scanning indices stop while pointing to the same element, i.e., $i = j$, the value they are pointing to must be equal to p . Thus, we have the sub-array partitioned, with the split position $s = i = j$:

Compiled: November 2018 Last update: January 2023 Page 95

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

We can combine the last case with the case of crossed-over indices ($i > j$) by exchanging the pivot with $A[j]$ whenever $i \geq j$.

Run Time Analysis of Quicksort

i. Worst-case partitioning

The running time of QUICKSORT is dominated by the time spent in the partition procedure. Each time the PARTITION procedure is called, a pivot element is selected, and this element is never included in any future recursive calls to QUICK-SORT and PARTITION.

Thus, there can be at most n calls to PARTITION over the entire execution of the quicksort algorithm.

One call to PARTITION takes $O(1)$ time plus an amount of time that is proportional to the number of iterations of the loop.

The partitioning costs $2(n)$ time. Since the recursive call on an array of size 0 just returns,

$T(0) = 2(1)$, and the recurrence for the running time is

$$\begin{aligned} T(n) &= T(n-1) + T(0) + 2(n) \\ &= T(n-1) + 2(n) \end{aligned}$$

Intuitively, if we sum the costs incurred at each level of the recursion, we get an arithmetic series (which evaluates to $2(n^2)$). Thus, if the partitioning is maximally unbalanced at every recursive level of the algorithm, the running time is $2(n^2)$ i.e. $O(n^2)$

Compiled: November 2018 Last update: January 2023 Page 96

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

Therefore the worst-case running time of quicksort is $2(n^2)$, which occurs when the input array is already completely sorted

ii. Best-case partitioning

In the most even possible split, PARTITION produces two sub-problems, each of size no more than $n/2$, since one is of size $\lfloor n/2 \rfloor$ and one of size $\lfloor n/2 \rfloor - 1$. In this case, quicksort runs much faster. The recurrence for the running time is then

$$T(n) \leq 2T(n/2) + 2(n),$$

which has the solution $T(n) = O(n \lg n)$

Thus, the equal balancing of the two sides of the partition at every level of the recursion produces an asymptotically faster algorithm.

Compiled: November 2018 Last update: January 2023 Page 97

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

5.3 Binary Search Tree

A Binary Search Tree (BST), also called ordered binary tree, has the following property:

For each node n of the tree,

- o all values stored in its left sub-tree (the tree whose root is the left child) are less than value v stored in n , and
- o all values stored in the right sub-tree are greater than v .

NOTE: Storing multiple copies of the same value in the same tree is avoided. An attempt to do so can be treated as an error.

The meaning of “less than” or “greater than” depend on the type of values stored in the tree: It is “ $<$ ” and “ $>$ ” for numerical values and alphabetical order in the case of strings.

Below is a representation of binary search tree.

The keys in a binary search tree are always stored in such a way as to satisfy the binary-search-tree property:

Let x be a node in a binary search tree.

- o If y is a node in the left sub-tree of x , then $y.key \leq x.key$.
- o If y is a node in the right sub-tree of x , then $y.key \geq x.key$.

Compiled: November 2018 Last update: January 2023 Page 98

CSC315 – Algorithms & Complexity Analysis – Orhionkpaiyo, B. C

For each node x it encounters, it compares the key k with $x.key$.

- o If the two keys are equal, the search terminates.
- o If k is smaller than $x.key$, the search continues in the left sub-tree of x , since the binary-search tree property implies that k could not be stored in the right sub-tree.
- o Symmetrically, if k is larger than $x.key$, the search continues in the right sub-tree.

The nodes encountered during the recursion form a simple path downward from the root of the tree, and thus the running time of TREE-SEARCH is $O(h)$, where h is the height of the tree.

□ The height of a tree is the number of nodes in the longest path from the root node to a leaf node.

□ In other words, if the depth of a binary search tree is n nodes so the running time of the algorithm will be $O(n)$.

However, for balanced binary tree, it is logarithmic running time. A tree is balanced if, for each node, the node’s sub-trees have the same height or have heights that differ by 1. For a balanced tree with n nodes, the height is $O(\log_2 n)$ which gives a worst-case time complexity that is logarithmic ($O(\log_2 n)$)

Exercises

1. Differentiate between Binary Search and Binary Search Tree

2. Solve the following recurrences using the master theorem

(a). $T(n) = 3T(n/4) + n$.

(b). $T(n) = T(n/2) + c$

(c). $T(n) = T(n-1) + 1$

Compiled: November 2018 Last update: January 2023 Page 99